

Università degli Studi di Milano

FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA

GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN

INFORMATICA MUSICALE

SVILUPPO DI UN PROCESSORE AUDIO PER IL MATCHING
AUTOMATICO DELL'EQUALIZZAZIONE

Relatore: Prof. Giorgio Presti

Elaborato Finale di:
Alessio Zampierolo
Matr. Nr. 09886A

ANNO ACCADEMICO 2024-2025

Ringraziamenti

È abbastanza assurdo e bello come, finito di scrivere tutto questo elaborato, mi ritrovi bloccato a cercare quattro parole in croce per ringraziare chi mi è stato vicino in questi anni. Probabilmente le parole non basteranno mai, quindi preferisco prendermi l'impegno di imparare a dimostrarvi la mia gratitudine nel tempo.

Intanto, però, è doveroso mettere nero su bianco i nomi dei diretti interessati. Quindi grazie a mia mamma Vania, a mio papà Massimo, a mia sorella Samantha, ai miei nipotini Alessandra e Andrea, al mio ragazzo Marco, a salvoubeila (Salvo, Ube e Ila) e, ovviamente, a momento besce (Alice, Pive, Andrea Testa, Pitte, Simo e Salvo).

Sommario (Abstract)

Il presente elaborato espone la progettazione e l'implementazione in linguaggio C++ di un processore audio real-time, sviluppato tramite il framework JUCE, concepito per ottimizzare e automatizzare il bilanciamento timbrico nella produzione musicale. L'elaborato nasce dall'esigenza di abbattere i tempi di produzione durante la fase di pre-mix e di stesura delle demo, fornendo all'utente un punto di partenza sonoro solido e oggettivo.

Come comunemente riportato nei testi di riferimento, l'equalizzazione rappresenta lo strumento principale per la scultura del timbro e la riduzione del mascheramento acustico [1]. Tuttavia, il raggiungimento di un bilanciamento ottimale richiede competenze e sensibilità uditive che non sempre sono a disposizione degli utenti meno esperti. Per questo motivo, l'utilizzo di tracce di riferimento (reference track) costituisce una pratica standard per fornire un traguardo sonoro oggettivo [1, 2]. L'equalizzatore grafico, grazie alla sua interfaccia semplice a frequenze fisse, risulta lo strumento più diretto per questo scopo.

Tuttavia, l'architettura classica degli equalizzatori grafici a filtri in cascata soffre di un limite matematico intrinseco: l'interazione tra le bande adiacenti (*crosstalk* o *ripple*). A causa di tale fenomeno, la risposta in frequenza reale del sistema non corrisponde alla somma algebrica dei guadagni visivamente impostati dall'utente [3]. Per risolvere questo problema, il motore Digital Signal Processing (DSP) del plugin integra una soluzione matematica basata su matrici di interazione, sfruttando la *self-similarity* dei filtri del secondo ordine [4]. Tale implementazione calcola i reali guadagni di comando (command gain) a partire da quelli impostati dall'utente (user gain), garantendo una corrispondenza esatta tra la curva di equalizzazione visiva e quella effettivamente applicata al segnale.

Risolto il problema dell'imprecisione dell'equalizzatore grafico, il motore matematico è stato esteso introducendo un sistema di automazione *cross-adaptive* [5]. Nello specifico, il plug-in analizza il profilo in frequenza di un segnale di ingresso (target) e di uno di riferimento (reference). Questa misurazione avviene su un arco di tempo di alcuni secondi per ottenere una stima stabile e affidabile. Successivamente, dopo aver convertito i segnali in mono e allineato i loro volumi (normalizzazione RMS), il sistema confronta le energie sulle singole bande ISO e applica in automatico la curva di equalizzazione necessaria.

Il risultato è uno strumento in grado di emulare le *best practices* del mixaggio [2], modificando autonomamente i parametri dell'equalizzatore grafico al fine di far convergere il profilo timbrico del target verso quello della reference.

Indice

Ringraziamenti

Sommario (Abstract)

Indice

1	Introduzione	1
1.1	Il contesto della produzione musicale e l'equalizzazione grafica	1
1.2	Il problema: l'interazione tra le bande a frequenza fissa	2
1.3	La soluzione e l'estensione: correzione matriciale e automazione	4
1.4	Organizzazione dell'elaborato	5
2	Stato dell'Arte e Background Teorico	7
2.1	L'errore di approssimazione (ripple) negli equalizzatori grafici a cascata . .	7
2.2	Soluzioni matematiche: la matrice di interazione (Abel, Berners, Välimäki)	8
2.3	Strumenti esistenti: panoramica sui processori commerciali di Graphic e Matching EQ	11
2.4	Metodologie Cross-Adaptive e misurazione dell'energia per bande (normalizzazione, RMS)	14
3	Tecnologie, Strumenti e Teoria dei Filtri	16
3.1	Il framework JUCE per il Digital Signal Processing	16
3.2	La libreria Eigen per l'algebra lineare e il calcolo matriciale	17
3.3	Tipologie di filtri utilizzate: teoria naive	18
4	Progettazione e Implementazione del Software	21
4.1	Organizzazione architetturale e separazione logica	21
4.1.1	GainMatrixCalculator: il modulo matematico	22
4.1.2	FilterBank: applicazione dei filtri di equalizzazione	23
4.1.3	AnalysisFilterBank: analisi energetica delle bande frequenziali . . .	24
4.2	Modulo di equalizzazione grafica	25

INDICE

4.2.1	Il percorso del segnale audio	25
4.2.2	Gestione dei parametri e aggiornamento dei filtri	26
4.3	Modulo di matching	27
4.3.1	Fase di learn	27
4.3.2	Fase di match	28
4.4	Interfaccia grafica	29
5	Test e Validazione	31
5.1	Metodologia e strumenti di misurazione spettrale	31
5.2	Validazione della matrice di interazione: coerenza tra user gain e risposta in frequenza effettiva	32
5.3	Valutazione dell'algoritmo di matching: misurazione della convergenza spettrale tra target e reference	35
6	Conclusioni e Sviluppi Futuri	39
6.1	Conclusioni sui risultati ottenuti	39
6.2	Sviluppi futuri	40
	Bibliografia	42

Capitolo 1

Introduzione

1.1 Il contesto della produzione musicale e l’equalizzazione grafica

Negli ultimi decenni l’abbattimento dei costi di hardware e software ha innescato una vera e propria “democratizzazione della tecnologia”, trasferendo la produzione musicale negli *home studio* [6]. Oggi è prassi comune comporre e finalizzare un brano interamente tra le mura di casa, dove nasce spesso l’esigenza di far suonare bene una demo o pre-produzione nel minor tempo possibile. Tuttavia, avere accesso agli strumenti non basta. Come osservato da Leyshon [6], la tecnologia ha reso disponibile a tutti un processo che in passato era accessibile solo “attraverso le strutture e le competenze fornite da uno studio di registrazione”. Proprio per compensare questa mancanza di esperienza tecnica e di un orecchio allenato, scaturisce la ricerca di processori che garantiscano risultati veloci e forniscano un vero e proprio supporto uditivo a chi muove i primi passi nel mixaggio.

Per intervenire sul suono, l’equalizzatore è universalmente riconosciuto come lo strumento più importante. Come sottolineato da Izhaki [1], non si tratta solo di uno strumento da usare nella produzione creativa, ma del mezzo principale per alterare il bilanciamento timbrico e mitigare il mascheramento acustico (*masking*), dando a ogni elemento il giusto spazio.

Nelle Digital Audio Workstation (DAW) moderne, i processori spettrali si dividono in due rami: gli equalizzatori parametrici, che offrono un controllo chirurgico e libero su ogni singolo filtro, e gli equalizzatori grafici (un esempio è mostrato in Figura 1). Questi ultimi si basano su un banco di filtri a frequenze centrali fisse, solitamente allineate alla ripartizione logaritmica dello standard dell’International Organization for Standardization (ISO) [2].

Grazie a questa architettura rigida ma intuitiva, l’equalizzatore grafico si comporta da vero e proprio “pannello di controllo” visivo dello spettro udibile. L’assenza di parametri

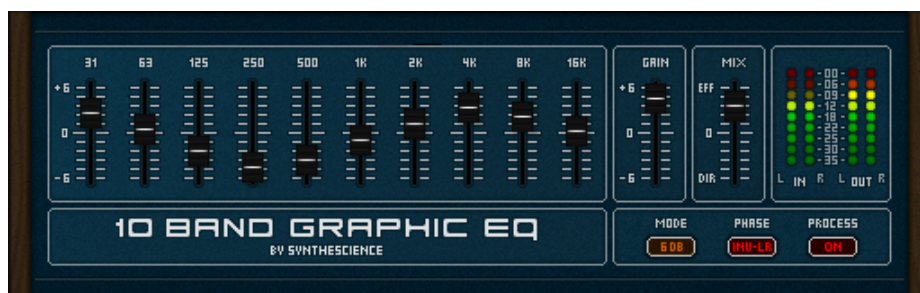


Figura 1: Esempio di interfaccia di un equalizzatore grafico digitale (plugin Voxengo Marvel GEQ).

di frequenza da ricercare e configurare manualmente rende l'interfaccia estremamente immediata, rendendo questo processore la scelta perfetta per interventi timbrici veloci e su larga scala.

Inoltre, lavorare puramente “a orecchio” in stanze spesso non trattate acusticamente può rivelarsi ingannevole. Per ovviare al problema, l'industria musicale si affida abitualmente all'ascolto comparato con brani commerciali di alta qualità, le cosiddette *reference tracks* [1]. Unire un “obiettivo” sonoro oggettivo e affidabile (la *reference*) ad uno strumento di correzione rapido come l'EQ grafico, costituisce l'esigenza pratica da cui prende spunto il presente elaborato.

1.2 Il problema: l'interazione tra le bande a frequenza fissa

A primo impatto, l'interfaccia a fader dell'equalizzatore grafico sembra garantire all'utente un controllo totale: la curva disegnata dai cursori dovrebbe rappresentare fedelmente l'alterazione timbrica applicata al segnale. In realtà, dietro questa apparente semplicità, si nasconde un limite strutturale ben noto nell'ambito dell'elaborazione dei segnali digitali.

Come analizzato da Välimäki e Reiss [3], la risposta in frequenza complessiva di un equalizzatore grafico implementato tramite filtri in cascata (o in parallelo) raramente corrisponde alla semplice somma algebrica dei valori impostati dall'utente. Il problema risiede nella natura stessa dei filtri digitali del secondo ordine (biquad). Nei processori a bande fisse si utilizzano tipicamente filtri a campana (*peaking filters*) che, a differenza dei filtri ideali “a mattone” (*brickwall*), presentano code spettrali (*skirts*) che si estendono oltre la propria frequenza centrale [4].

Dal punto di vista pratico, ciò implica che le bande adiacenti interagiscono tra loro in modo costruttivo o distruttivo. Se, ad esempio, l'utente solleva un gruppo di fader per ottenere un guadagno uniforme di +10 dB, l'energia delle code spettrali tende a sommarsi.

Il risultato effettivo non sarà una linea retta a +10 dB, ma una curva ondulata in cui i picchi, in corrispondenza delle frequenze centrali, possono raggiungere o superare i +14 dB, a causa dell'accumulo di energia proveniente dalle bande vicine [4, 3] come evidenziato in Figura 2.

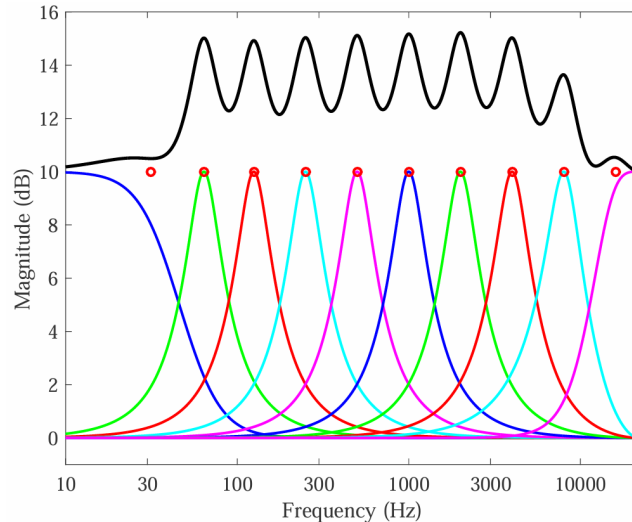


Figura 2: Risposta in frequenza di un equalizzatore grafico a bande d'ottava implementato tramite struttura a cascata (linea spessa) e contributo dei singoli filtri (curve colorate, $Q = 1.41$). I cerchi rossi indicano i command gain, impostati a +10 dB. Estratto da [3].

Questo fenomeno di interazione tra i filtri è noto come *crosstalk* tra bande ed è la causa del cosiddetto errore di *ripple* (ondulazione). Nel tentativo di tracciare una curva fluida, il sistema restituisce in realtà una risposta in frequenza irregolare, caratterizzata da picchi e avvallamenti non desiderati [2].

Per analizzare più formalmente il problema, Abel e Berners [4] introducono una distinzione fondamentale tra user gain, ovvero il valore impostato visivamente dall'utente sul fader, e command gain, cioè il guadagno effettivamente applicato al filtro. Negli equalizzatori grafici tradizionali questi due valori coincidono: il sistema applica direttamente al filtro il valore selezionato dall'utente. È proprio questo approccio diretto a generare l'errore di sovrapposizione, creando l'illusione per cui la curva visualizzata non corrisponde esattamente alla risposta sonora percepita.

Questo limite di approssimazione rappresenta il punto di partenza per un possibile miglioramento del sistema. L'obiettivo del presente elaborato è infatti quello di affrontare il problema dell'interazione tra bande introducendo una correzione basata su matrici di interazione. Tale approccio consente di ottenere una risposta in frequenza più fedele ai valori desiderati e costituisce una base solida per l'estensione successiva verso tecniche

di matching automatico, in cui una traccia target viene adattata spettralmente sulla base di una reference.

1.3 La soluzione e l'estensione: correzione matriciale e automazione

Per migliorare allora l'imprecisione tipica degli equalizzatori grafici classici, questo progetto implementa un metodo di compensazione matematica, al quale viene successivamente affiancato un sistema di automazione per il matching dell'equalizzazione.

Il primo passaggio è stato quello di rendere indipendente l'interfaccia visiva dall'elaborazione audio vera e propria. Sfruttando quindi gli studi sulle matrici di interazione [4, 3], il plugin non invia più ai filtri i valori di guadagno "grezzi" scelti dall'utente (user gain), ma un algoritmo interno analizza la posizione di tutti i cursori e, tramite un calcolo basato su una matrice di interazione, ricava i guadagni di comando effettivi (command gain) necessari per rimuovere l'errore di *ripple*. In questo modo, l'equalizzatore grafico diventa uno strumento effettivamente preciso, dove la curva visibile a schermo coincide con la reale risposta in frequenza applicata al segnale (si veda la Figura 3).

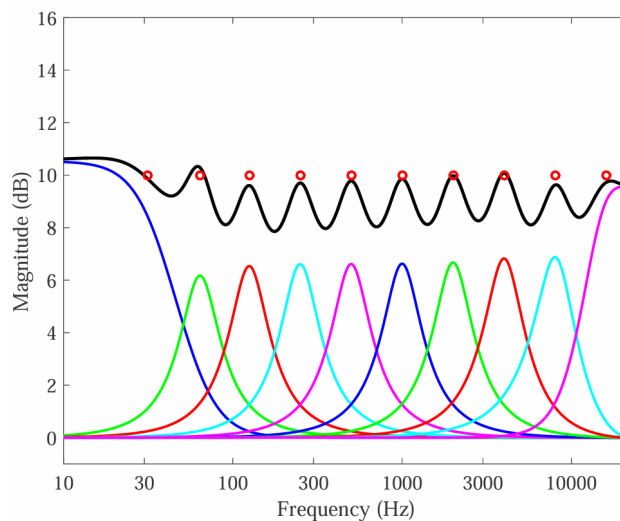


Figura 3: Risposta in frequenza dell'equalizzatore grafico a cascata una volta risolti i guadagni dei filtri tenendo conto dell'interferenza tra bande. I picchi dei filtri non corrispondono ai command gains (cerchi rossi). Estratto da [3].

Una volta garantita la precisione su questo aspetto, l'architettura è stata estesa introducendo un algoritmo per automatizzare il processo di matching EQ. L'idea di base è quella di permettere al plug-in di analizzare il bilanciamento spettrale di un segnale di

riferimento (la reference), per poi alterare la traccia in ingresso (il target) al fine di farla suonare il più simile possibile al riferimento.

Per ottenere un'analisi affidabile, specialmente per garantire una corretta analisi del contenuto energetico nelle basse frequenze, il sistema acquisisce l'audio in ingresso su una finestra temporale di circa cinque secondi. Successivamente, dopo una fase di pre-elaborazione in cui il segnale viene trasformato in mono e il volume generale viene normalizzato, il software calcola la differenza di energia RMS tra le singole bande dei due segnali tramite un banco di filtri di misurazione dedicato.

Una volta calcolata la differenza tra bande, il sistema non forza i valori “dietro le quinte”, ma va a “muovere” fisicamente i fader dell'interfaccia. Questo aggiornamento dei parametri visivi innesca a catena il calcolo matematico dei command gain per i filtri descritto prima, proprio come se ci fosse stato il cambiamento a livello degli user gain. Questo approccio fornisce all'utente un punto di partenza immediato, sul quale è poi possibile intervenire liberamente per apportare ulteriori modifiche manuali in base al proprio gusto.

1.4 Organizzazione dell'elaborato

L'elaborato è strutturato in sei capitoli, organizzati per descrivere progressivamente il percorso logico del progetto: dalla teoria matematica di base fino alla stesura del codice e ai test finali.

Nel Capitolo 2 (“Stato dell'Arte e Background Teorico”) si approfondiscono le basi fisiche e matematiche del problema. Verrà analizzato nel dettaglio l'errore di *ripple* negli equalizzatori a cascata e verrà spiegata la soluzione basata sulle matrici di interazione. Dopo una breve panoramica sui plug-in commerciali esistenti, il capitolo illustrerà la logica utilizzata per estrarre e confrontare l'energia tra le diverse bande, alla base dell'algoritmo di matching EQ.

Il Capitolo 3 (“Tecnologie, Strumenti e Teoria dei Filtri”) presenta gli strumenti informatici scelti per lo sviluppo del plug-in. Si parlerà del framework JUCE, utilizzato per la programmazione audio e DSP, e della libreria Eigen per la gestione dell'algebra lineare. Si darà poi spazio all'analisi dei filtri usati, mostrando il passaggio dai prototipi teorici ideali fino alla loro implementazione pratica in C++/JUCE.

Nel Capitolo 4 (“Progettazione e Implementazione del Software”) si entra nello specifico della scrittura del codice. In questa sezione verrà “smontata a pezzi” l'architettura del plugin, spiegando come le formule matematiche e le basi teoriche dei capitoli precedenti siano state tradotte in un processore audio informatico.

Il Capitolo 5 (“Test e Validazione”) ha l'obiettivo di dimostrare l'effettivo funzionamento del sistema. Verranno mostrati i grafici che provano l'efficacia della matrice di interazione (confrontando i cursori visivi con la reale risposta in frequenza) e si valuteranno le prestazioni dell'algoritmo di matching, misurando quanto effettivamente la traccia in ingresso riesca ad avvicinarsi alla reference.

Infine, il Capitolo 6 (“Conclusioni e Sviluppi Futuri”) tira le somme del lavoro svolto, commentando i risultati ottenuti e proponendo alcune prospettive di ricerca futura. Tra queste, viene esposto un prototipo MATLAB per estendere il processamento in ambito spaziale (Mid/Side) e l’introduzione di una maggiore libertà nella scelta delle scale frequenziali.

Capitolo 2

Stato dell'Arte e Background Teorico

2.1 L'errore di approssimazione (ripple) negli equalizzatori grafici a cascata

La maggior parte degli equalizzatori grafici digitali viene solitamente implementata adottando un'architettura a filtri in cascata. Da un punto di vista dell'elaborazione dei segnali, questo implica che il flusso audio attraversa in modo sequenziale un banco di filtri biquadratici, ciascuno accordato su una specifica frequenza centrale.

Matematicamente, la funzione di trasferimento complessiva di un equalizzatore grafico a cascata composto da M bande, indicata come $H_{\text{CGEQ}}(z)$, è definita dal prodotto delle funzioni di trasferimento $H_m(z)$ dei singoli filtri [3]:

$$H_{\text{CGEQ}}(z) = G_0 \prod_{m=1}^M H_m(z) \quad (1)$$

dove G_0 rappresenta un eventuale guadagno globale, solitamente impostato a 1 (0 dB) per non generare un offset sul volume generale del segnale in transito.

Idealmente, ogni filtro $H_m(z)$ dovrebbe presentare un modulo pari al guadagno desiderato G_m all'interno della propria banda, e un modulo pari a $\sqrt{G_m}$ ai margini della banda stessa (le *band edges*). Di conseguenza, alla frequenza di incrocio (*crossover frequency*) tra due bande adiacenti, ipotizzando di impostare il medesimo guadagno $G_m = G_{m+1} = G$, il prodotto dei moduli dei due filtri garantirebbe una risposta perfetta [3]:

$$|H_m(e^{j\omega_{l,m}}) H_{m+1}(e^{j\omega_{u,m+1}})| = G \quad (2)$$

Per la realizzazione di questa architettura, il filtro corrispondente alla banda più bassa e quello corrispondente alla banda più alta sono tipicamente implementati come filtri shelving del secondo ordine, mentre per tutte le restanti bande intermedie si usano filtri

peak/notch [3]. Tuttavia, nella pratica, i filtri del banco si influenzano a vicenda, modificando il guadagno effettivo. Questo causa una mutua interazione tra le bande (*crosstalk*), facendo in modo che il guadagno totale differisca dal valore teorico atteso [3, 4].

Dato che i filtri operano in cascata, il prodotto delle loro funzioni di trasferimento si traduce, nel dominio logaritmico dei decibel (dB), in una somma algebrica delle risposte in frequenza. Questa sovrapposizione energetica è la causa del *gain buildup* (accumulo di guadagno). Per fare un esempio, assegnando a tutti i filtri del banco un medesimo guadagno operativo (ad esempio +10 dB), l'interazione costruttiva delle code porta a un'alterazione evidente, dove la curva di risposta reale non risulterà piatta a 10 dB, ma presenterà picchi che possono raggiungere e superare i +15 dB in corrispondenza delle frequenze centrali [3].

Oltre all'accumulo sui picchi, la risposta in frequenza tra una banda e l'altra subisce delle flessioni verso il basso, originando un errore di *ripple*, la cui ampiezza picco-picco è proporzionale ai guadagni applicati. Un tentativo apparentemente intuitivo per risolvere questa sovrapposizione potrebbe consistere nell'incrementare il fattore di merito (Q) dei filtri, restringendone la campana. In realtà, gli studi dimostrano che questo approccio è controproducente, in quanto l'assottigliamento delle bande, pur limitando il *gain buildup* sui picchi, causa un avvallamento ancora più evidente del guadagno nelle frequenze di incrocio, incrementando ulteriormente l'ondulazione del *ripple* [3].

Da questa analisi emerge quindi un limite strutturale nell'architettura a cascata: se i guadagni visivi impostati dall'utente (user gain) venissero utilizzati direttamente come parametri di controllo per i filtri, il sistema genererebbe uno scostamento tra l'interfaccia grafica e l'effettiva equalizzazione del segnale [4].

2.2 Soluzioni matematiche: la matrice di interazione (Abel, Berners, Välimäki)

Per risolvere dunque il problema del *gain buildup* e rendere l'equalizzatore grafico uno strumento visivamente preciso, bisogna “abbandonare” l'idea dell'approccio di controllo diretto (user gain $\rightarrow$ filtri). Una possibile soluzione è stata formalizzata dagli studi di Abel e Berners [4] e successivamente approfondita da Välimäki [3]. L'idea è quella di introdurre un livello di calcolo matriciale tra l'interfaccia usata dall'utente e i filtri veri e propri, pre-calcolando il valore dell'interazione.

Come descritto nel paragrafo precedente, nel dominio dei decibel le risposte in frequenza dei filtri in cascata si sommano. Di conseguenza, il guadagno effettivo misurato a una specifica frequenza centrale f_i non dipende unicamente dal filtro accordato su quella frequenza, ma dalla somma delle “code” di tutti i filtri del banco.

L'idea alla base è che le risposte in ampiezza dei singoli filtri possano essere utilizzate come funzioni di base per stimare la risposta totale dell'equalizzatore. Come dimostrato da

Välimäki [3], se consideriamo un equalizzatore a M bande, la stima dell'ampiezza effettiva misurata in uscita può essere descritta dalla seguente equazione matriciale:

$$\hat{\mathbf{h}} = \mathbf{B} \mathbf{g} \quad (3)$$

dove:

- $\hat{\mathbf{h}}$ è il vettore di M elementi che rappresenta la stima della risposta in ampiezza reale misurata alle frequenze centrali
- $\mathbf{g}$ è il vettore di M elementi che contiene i guadagni desiderati (impostati visivamente tramite i fader, gli user gain)
- $\mathbf{B}$ è la matrice di interazione $M \times M$, i cui elementi $b_{i,j}$ indicano l'interferenza tra le bande. In parole semplici, il valore $b_{i,j}$ indica quanti decibel si aggiungono “per sbaglio” sulla frequenza i a causa della coda del filtro j .

Nel caso specifico di un equalizzatore grafico ad ottave, lo standard internazionale ISO 266 definisce un banco di $M = 10$ frequenze centrali: 31.5, 63, 125, 250, 500, 1000, 2000, 4000, 8000 e 16000 Hz. Di conseguenza, la matrice $\mathbf{B}$ sarà una matrice 10×10 .

All'interno della matrice $\mathbf{B}$, la diagonale principale ($b_{1,1}, b_{2,2}, \dots, b_{10,10}$) contiene valori molto vicini a 1, poiché un filtro accordato a f_i ha la sua massima influenza esattamente su f_i . I valori al di fuori della diagonale, invece, rappresentano l'interferenza. Ad esempio, l'elemento $b_{6,7}$ rappresenta il guadagno generato dal filtro a 2000 Hz misurato sulla frequenza di 1000 Hz.

Dato che i filtri su scala logaritmica presentano una struttura approssimativamente simmetrica (proprietà chiamata *self-similarity*), i coefficienti tendono a diminuire man mano che ci si allontana dalla diagonale principale, riflettendo l'attenuazione naturale delle code spettrali [4].

Questa prima equazione dimostra matematicamente l'accumulo di energia: mandando ai filtri direttamente i valori scelti dall'utente ($\mathbf{g}$), la risposta effettiva in uscita ($\hat{\mathbf{h}}$) risulta alterata dalla matrice $\mathbf{B}$, e pertanto non coincide con $\mathbf{g}$.

L'obiettivo diventa quindi quello di portare la curva in uscita a coincidere con le impostazioni dell'utente. Per farlo, è necessario calcolare un nuovo vettore di guadagni “ottimizzati”, indicato come $\mathbf{g}_{\text{opt}}$ (implementato nel progetto come command gain), da inviare effettivamente ai filtri.

L'equazione per ottenere la curva desiderata diventa quindi:

$$\mathbf{g} = \mathbf{B} \mathbf{g}_{\text{opt}} \quad (4)$$

La rappresentazione del sistema per l'equalizzatore a 10 bande assume la seguente forma:

$$\begin{pmatrix} g_{31.5} \\ g_{63} \\ \vdots \\ g_{16k} \end{pmatrix} = \begin{pmatrix} b_{1,1} & b_{1,2} & \cdots & b_{1,10} \\ b_{2,1} & b_{2,2} & \cdots & b_{2,10} \\ \vdots & \vdots & \ddots & \vdots \\ b_{10,1} & b_{10,2} & \cdots & b_{10,10} \end{pmatrix} \begin{pmatrix} g_{\text{opt},31.5} \\ g_{\text{opt},63} \\ \vdots \\ g_{\text{opt},16k} \end{pmatrix} \quad (5)$$

A questo punto, la soluzione del problema consiste in un'operazione di algebra lineare. L'utente, muovendo i fader sull'interfaccia, imposta i valori del vettore $\mathbf{g}$. Il sistema deve quindi determinare quali valori inserire nel vettore $\mathbf{g}_{\text{opt}}$, da fornire ai filtri per ottenere esattamente quella curva.

Dato che si tratta di una matrice invertibile, basta calcolare la matrice inversa $\mathbf{B}^{-1}$ e moltiplicarla per il vettore delle impostazioni utente:

$$\mathbf{g}_{\text{opt}} = \mathbf{B}^{-1} \mathbf{g} \quad (6)$$

Come dimostrato da Välimäki [3], fornendo ai filtri il vettore dei guadagni di comando $\mathbf{g}_{\text{opt}}$ calcolato in questo modo, si compensa la sovrapposizione delle bande nell'architettura a cascata. L'interazione tra le code, che causa l'accumulo di energia, viene in questo modo prevista e compensata matematicamente.

Di seguito è riportata la matrice di interazione $\mathbf{B}$ calcolata per il banco a 10 bande, che verrà implementata in modo statico (hard-coded) all'interno del codice sorgente:

$$\mathbf{B} = \begin{bmatrix} 0.80 & 0.23 & 0.02 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0.19 & 1 & 0.21 & 0.04 & 0.01 & 0 & 0 & 0 & 0 & 0 \\ 0.04 & 0.21 & 1 & 0.20 & 0.04 & 0.01 & 0 & 0 & 0 & 0 \\ 0.01 & 0.04 & 0.20 & 1 & 0.20 & 0.04 & 0.01 & 0 & 0 & 0 \\ 0 & 0.01 & 0.04 & 0.20 & 1 & 0.20 & 0.04 & 0.01 & 0 & 0 \\ 0 & 0 & 0.01 & 0.04 & 0.20 & 1 & 0.20 & 0.04 & 0.01 & 0 \\ 0 & 0 & 0 & 0.01 & 0.04 & 0.20 & 1 & 0.20 & 0.03 & 0 \\ 0 & 0 & 0 & 0 & 0.01 & 0.04 & 0.21 & 1 & 0.18 & 0.01 \\ 0 & 0 & 0 & 0 & 0 & 0.01 & 0.06 & 0.25 & 1 & 0.10 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.01 & 0.14 & 0.94 \end{bmatrix} \quad (7)$$

Come descritto da Välimäki [3], i coefficienti di questa matrice sono stati calcolati analiticamente (sulla base dei filtri usati) impostando tutti i guadagni dell'equalizzatore a +10 dB, misurando l'ampiezza alle dieci frequenze centrali e dividendo i valori ottenuti per 10.

Infine, per minimizzare ulteriormente l'ondulazione residua tra i punti di comando, il sistema sfrutta il parametro di guadagno globale G_0 : impostando questo valore alla mediana dei guadagni scelti dall'utente, si applica un offset all'intero vettore degli user gain che stabilizza ulteriormente la curva [3].

2.3 Strumenti esistenti: panoramica sui processori commerciali di Graphic e Matching EQ

Per quanto riguarda i plug-in già in commercio, si può notare come l'equalizzazione grafica e il matching EQ vengano spesso trattati come due mondi separati, ciascuno con i propri compromessi, limiti e soluzioni.

Relativamente agli equalizzatori grafici, la gestione dell'interazione tra le bande è affrontata dai produttori con strategie diverse in base all'obiettivo sonoro voluto. Molti vogliono imitare volutamente il comportamento dei filtri di famosi equalizzatori analogici, altri invece cercano strategie per affrontare e risolvere il problema di interazione tra le bande.

Molti plugin adottano l'architettura *proportional-Q*. In processori come il Waves API 560 [7] o nella componente "Classic" del processore Waves GEQ [8], la larghezza della campana del filtro (il fattore Q) si restringe automaticamente all'aumentare o al diminuire del guadagno, riducendo così l'area di sovrapposizione alle impostazioni più elevate (come illustrato in Figura 4).

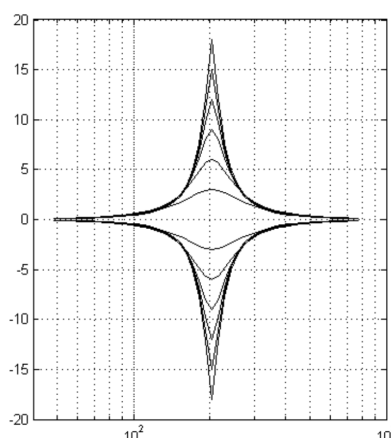


Figura 4: Esempio di comportamento *proportional-Q*: la larghezza del filtro si restringe all'aumentare del guadagno. Grafico tratto dalla documentazione di [8].

Per avere ancora più precisione vengono spesso utilizzate tipologie di filtri alternative. Come nel caso della variante "Modern" dello stesso processore Waves GEQ, che abbandona il comportamento proporzionale in favore di speciali filtri Flat-Top. Come descritto nel manuale [8], questa tecnologia elimina l'interazione creando risposte piatte a "gradino" (*stair-stepped*) o a *plateau*. Questo approccio risolve il problema visivo della sovrapposizione, ma lo fa alterando la forma stessa dell'equalizzazione, che perde la classica e naturale curva a campana in favore di un profilo più squadrato (si veda il confronto in Figura 5).

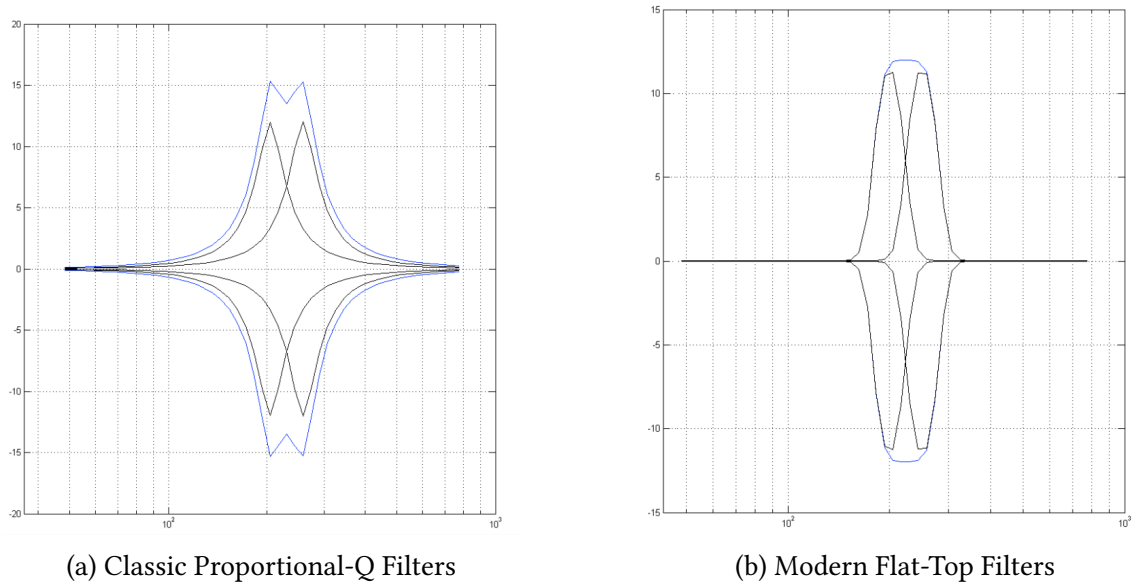


Figura 5: Confronto tra filtri *Proportional-Q* e filtri *Flat-Top*. Immagini tratte dalla documentazione di [8].

Invece il meccanismo basato su matrice di interazione trattato in questo elaborato [3] rappresenta quindi un'altra via: permette di annullare l'interazione calcolando matematicamente i guadagni pre-compensati, mantenendo però intatta la topologia e il suono dei classici filtri IIR del secondo ordine (peaking filters).

Si può inoltre notare come queste logiche matriciali di compensazione siano state originariamente formalizzate nei lavori di Jonathan S. Abel e David P. Berners [4], ricercatori storicamente legati alla ricerca e sviluppo per Universal Audio. Ciò dimostra come questo approccio derivi direttamente da una ricerca industriale e accademica di grande importanza.

Spostando l'attenzione sull'automazione del matching EQ, i processori commerciali adottano una pipeline operativa spesso simile, strutturata in fasi sequenziali. Il processo inizia con una fase di apprendimento (*learn* o *capture*), durante la quale l'algoritmo analizza il comportamento frequenziale di una traccia di riferimento (*reference*) e quello della traccia in ingresso. Ottenuti i due profili, il sistema calcola la curva di differenza necessaria per far combaciare lo spettro in ingresso con quello target.

È nella successiva fase di applicazione di questa curva che i software differiscono per il risultato offerto all'utente. Ad esempio, processori come FabFilter Pro-Q 3 usano un modello parametrico adattivo. Questo approccio, come descritto nella documentazione ufficiale [9], traduce la curva di differenza in una serie di nodi parametrici. Al termine dell'analisi, l'algoritmo genera automaticamente nuove bande di equalizzazione per approssimare lo spettro di riferimento (Figura 6).



Figura 6: Interfaccia del plugin FabFilter Pro-Q 3 durante l'utilizzo della funzione di EQ Match. Immagine tratta dalla documentazione ufficiale [9].

Oppure, un approccio alternativo come quello del processore iZotope Ozone 11 [10], non genera nodi parametrici discreti visibili all'utente ma applica direttamente l'intera curva di differenza in modo continuo. L'interazione dell'utente avviene su parametri globali, dove è possibile regolare l'intensità complessiva dell'equalizzazione (*amount*) o attenuare la precisione dei picchi per ottenere un profilo più morbido (*smoothing*), come mostrato dall'interfaccia in Figura 7.



Figura 7: Interfaccia del plugin iZotope Ozone 11 nella sezione Match EQ. Immagine tratta dalla documentazione ufficiale [10].

In questo contesto, il processo sviluppato in questo elaborato usa la pipeline di estrazione spettrale del matching EQ applicandola ad un'interfaccia grafica a ottave fisse. L'utilizzo della matrice di compensazione permette poi all'algoritmo di automazione di calcolare la posizione dei 10 fader in modo che la curva reale applicata dai filtri coincida esattamente con la differenza spettrale misurata.

Ciò offre all'utente un punto di partenza automatizzato su una topologia standard, combinando quindi l'intuitività e semplicità visiva di un equalizzatore grafico tradizionale con l'uso dei classici filtri IIR.

2.4 Metodologie Cross-Adaptive e misurazione dell'energia per bande (normalizzazione, RMS)

La parte di automazione per il matching EQ si compone di due fasi operative principali: la fase di acquisizione (learn), in cui il codice memorizza i campioni audio del segnale in ingresso (target) e di quello di riferimento (reference, attraverso un sidechain se presente), e la successiva fase di calcolo (match).

I buffer ausiliari utilizzati durante la fase di learn vengono acquisiti ed elaborati direttamente in mono e hanno una durata di 5 secondi. Una volta memorizzati i campioni, il sistema procede alla normalizzazione dei due buffer. Questo passaggio è necessario per non falsare l'analisi del bilanciamento tonale. Nell'ambito dell'equalizzazione autonoma multitraccia (ma in generale in tante operazioni sonore che si basano sull'utilizzo e confronto di più tracce, tra cui le reference), compensare le discrepanze di volume globale (*perceptual loudness*) assicura che il sistema valuti esclusivamente le differenze spettrali e timbriche tra le tracce, escludendo l'influenza del volume dei segnali [2].

Per calcolare l'energia presente all'interno delle 10 bande dell'equalizzatore, il sistema sfrutta un'analisi nel dominio del tempo. I segnali normalizzati vengono fatti scorrere attraverso un banco di filtri di analisi accordati sulle medesime frequenze centrali standard ISO usate dal modulo di equalizzazione grafica. I filtri utilizzati in questa fase di misurazione differiscono ovviamente da quelli impiegati per la successiva equalizzazione del segnale. Mentre l'equalizzatore grafico utilizza filtri low-shelf, peaking e high-shelf per sommare o sottrarre guadagno in una specifica banda senza eliminare il resto dello spettro, l'algoritmo di analisi impiega un banco composto da filtri low-pass, band-pass e high-pass. In questo modo, l'energia misurata in ciascuna banda deriva esclusivamente dalla regione frequenziale di interesse, escludendo le interferenze spettrali adiacenti (cosa che si cerca nel matching ma non nell'equalizzazione grafica).

Successivamente all'isolamento della banda, viene calcolata l'intensità media del segnale filtrato tramite il valore RMS (Root Mean Square). L'uso del calcolo RMS è una pratica consolidata nei sistemi di equalizzazione autonoma per stimare l'energia all'interno di specifiche regioni dello spettro [2]. Inoltre, l'utilizzo di questo valore è giustificato dalla sua stretta correlazione con la psicoacustica. A differenza dell'ampiezza di picco istantanea, il calcolo RMS restituisce un livello medio nel tempo che approssima in modo fedele l'effettiva intensità percepita dall'orecchio umano, fornendo una misurazione dell'ampiezza molto più realistica e coerente con il nostro apparato uditivo [11].

Matematicamente, la funzione di libreria (`tempBuffer.getRMSLevel(...)`) utilizzata per il calcolo estrae il livello RMS dell'intero blocco di campioni. Definendo $y_{f_c}[n]$ come l'array di campioni del buffer dopo che questo è stato processato dal filtro di analisi (centrato a f_c), l'energia $X(f_c)$ viene calcolata tramite l'equazione standard della radice quadrata media:

$$X(f_c) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (y_{f_c}[n])^2} \quad (8)$$

dove:

$y_{f_c}[n]$ rappresenta il valore dell'ampiezza del segnale al campione n -esimo, precedentemente isolato nella banda di interesse.

N è la lunghezza complessiva del buffer analizzato (espressa in numero di campioni).

L'algoritmo genera così due array di dieci valori (uno per il target e uno per la reference). La differenza in decibel tra questi due array determina l'esatto guadagno di compensazione da applicare ai fader dell'interfaccia utente. Infine, come descritto nei paragrafi precedenti, tali valori utente verranno tradotti dalla matrice di interazione del DSP per garantire che la curva finale misurata coincida con quella richiesta.

Capitolo 3

Tecnologie, Strumenti e Teoria dei Filtri

3.1 Il framework JUCE per il Digital Signal Processing



Figura 8: Logo del framework JUCE.

Il codice del progetto descritto in questo elaborato è stato sviluppato interamente in linguaggio C++ avvalendosi di JUCE (Figura 8), un framework cross-platform utilizzato per la creazione di applicazioni audio e plug-in [12].

Ciò che rende JUCE così utile deriva principalmente dal suo elevato livello di astrazione. JUCE funziona infatti da wrapper tra il codice sorgente e il sistema operativo, consentendo di gestire sia l'interfaccia grafica che il motore audio tramite un'unica code-base. Inoltre, il framework si fa carico di astrarre le specifiche tecniche dei diversi formati di plug-in esistenti (come VST3, Audio Unit e AAX). In questo modo, il programmatore può concentrarsi “solamente” sulla logica algoritmica di ciò che vuole creare, dando alla libreria il compito di interfacciarsi correttamente con le varie Digital Audio Workstation (DAW) [13].

Per quanto riguarda l'elaborazione dei segnali, questo progetto fa ampio uso del modulo dedicato `juce_dsp` [14]. Questa libreria fornisce un'architettura standardizzata e una

serie di algoritmi ottimizzati per il Digital Signal Processing (DSP), mettendo a disposizione classi fondamentali come processori FFT e blocchi per il filtraggio (come, ad esempio, la classe `juce::dsp::IIR::Filter`, largamente utilizzata).

Affidarsi alla struttura offerta da JUCE offre notevoli vantaggi in termini di prestazioni. Una caratteristica essenziale è la sua progettazione orientata alle applicazioni in tempo reale: le sue strutture dati sono implementate per operare in modo sicuro all'interno del metodo `processBlock` (il ciclo di audio callback), evitando allocazioni dinamiche di memoria che potrebbero causare interruzioni nel flusso audio (dropout). Inoltre, per garantire la maggior efficienza computazionale necessaria all'elaborazione simultanea svolta dai molteplici filtri usati, le classi di questo modulo sfruttano "dietro le quinte" ottimizzazioni vettoriali basate su istruzioni processore SIMD (Single Instruction Multiple Data) [14].

3.2 La libreria Eigen per l'algebra lineare e il calcolo matriciale

Per la gestione delle operazioni matematiche, il progetto integra Eigen, una libreria C++ open-source basata su template, specializzata nell'algebra lineare e nel calcolo matriciale [15]. Il vantaggio principale di Eigen sta nella sua natura header-only: non necessita di file binari precompilati o di complesse fasi di linking, quindi la libreria viene usata facilmente all'interno del codice con JUCE.

All'interno del progetto, l'utilizzo di Eigen lo si trova soprattutto nella classe dedicata `GainMatrixCalculator`, creata per l'implementazione del modello matematico di compensazione dei filtri. Nello specifico, l'algoritmo sfrutta le strutture dati fornite dalla libreria (come `Eigen::Matrix` ed `Eigen::Array`) per definire e allocare in memoria la matrice di interazione 10x10 dell'equalizzatore grafico, le frequenze centrali e i vettori dei guadagni.

L'efficienza della libreria è fondamentale nella fase di gestione dei parametri di equalizzazione. Durante l'utilizzo, l'utente imposta sull'interfaccia grafica i guadagni desiderati (gli user gain, gestiti nel resto del codice come normali array di float), ma per ricavare i guadagni effettivi da applicare ai filtri del DSP (i command gain), i valori inseriti dall'utente devono essere elaborati attraverso la matrice di interazione.

Per eseguire questo calcolo in modo ottimizzato, l'array di float in ingresso viene mappato direttamente in un formato vettoriale tramite la classe `Eigen::Map`. Questo approccio permette di tradurre i dati standard del C++ in un formato compatibile con Eigen. A questo punto, sfruttando le funzioni della libreria, l'algoritmo esegue il calcolo in tre passaggi:

- Calcola la media aritmetica dei guadagni utente per ottenere l'offset globale G_0 .
- Sottrae questo offset dal vettore dei guadagni originali.

- Esegue il prodotto matriciale tra il vettore risultante e la matrice di interazione.

Il risultato di quest'ultima operazione fornisce i command gain definitivi. L'intera transizione dai guadagni visivi a quelli reali è incapsulata all'interno di questa classe dedicata. Questo fa in modo che tutti i calcoli algebrici necessari ad annullare l'interazione tra le bande vengano eseguiti in modo rapido e efficiente.

3.3 Tipologie di filtri utilizzate: teoria naive

L'architettura standard per un equalizzatore grafico utilizza filtri a campana (peaking) per le bande centrali e filtri shelving per gli estremi dello spettro [3]. Seguendo un approccio speculare, la parte di analisi del progetto richiede filtri selettivi in frequenza (passa-banda, passa-basso e passa-alto) per isolare l'energia delle specifiche regioni.

Per tradurre questi concetti in codice, il progetto usa il modulo `juce::dsp`, i cui filtri digitali IIR derivano dalla discretizzazione tramite trasformata bilineare [14].

Per la creazione della parte di equalizzazione, il progetto sfrutta il filtro peaking per processare le 8 bande centrali dell'equalizzatore. Questo filtro aumenta o riduce l'energia attorno a una frequenza centrale intervenendo in modo limitato sul resto dello spettro. Il suo prototipo analogico a due poli, nella forma base, è definito dalla funzione di trasferimento $H_{PK}(s) = H_{LP}(s) - H_{HP}(s) = \frac{1-s^2}{s^2+2Rs+1}$, ricavabile praticamente dalla sottrazione delle risposte passa-basso (H_{LP}) e passa-alto (H_{HP}) [16]. La larghezza e l'altezza della risonanza dipendono dal fattore di smorzamento R , comunemente tradotto nel parametro di qualità Q , a cui è legato dalla relazione inversamente proporzionale $Q = \frac{1}{2R}$ [16], come mostrato in Figura 9.

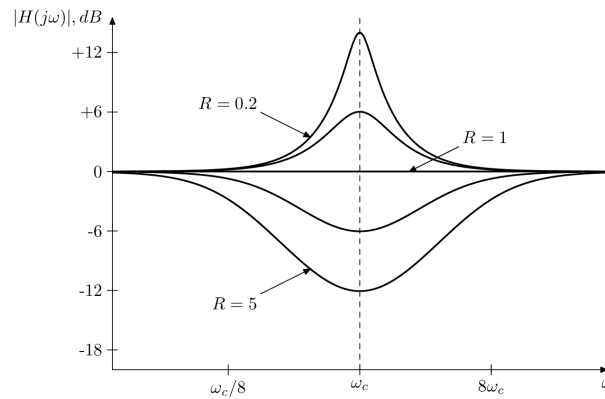


Figura 9: Risposta del filtro peaking al variare di R . Estratto da: [16].

```
coeffs = juce::dsp::IIR::Coefficients<float>::makePeakFilter(
    sampleRate, fc, Q, g );
```

Per le bande agli estremi dello spettro vengono invece impiegati i filtri shelving. Questi filtri modificano il guadagno in modo asimmetrico da una frequenza di taglio verso l'infinito. Un filtro shelving a due poli ideale (filtro del secondo ordine) si costruisce sommando il segnale originale a una sua versione filtrata e scalata. Nel dominio analogico continuo, per il low-shelf la funzione di trasferimento è $H_{LS}(s) = 1 + K \cdot H_{LP}(s)$, mentre per l'high-shelf si utilizza la formula $H_{HS}(s) = 1 + K \cdot H_{HP}(s)$, dove K rappresenta il guadagno e H_{LP}/H_{HP} identificano i rispettivi filtri passa-basso e passa-alto [16] (si veda la Figura 10).

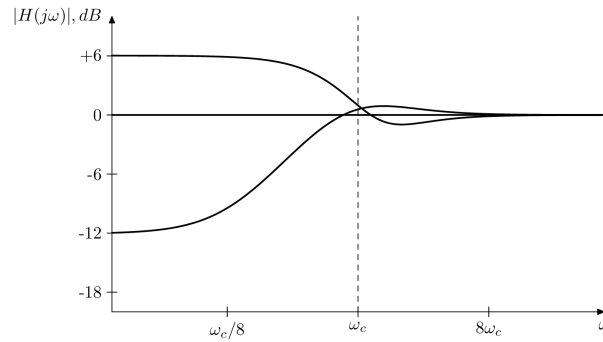


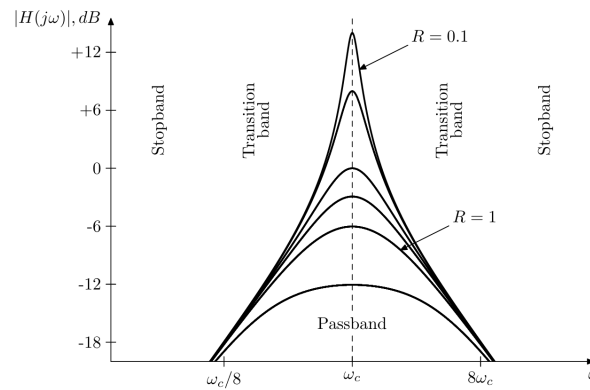
Figura 10: Risposta del filtro shelving al variare di K . Estratto da: [16].

Nota: queste equazioni descrivono il principio teorico di base, definendone l'approccio in modo "naive" [16]. L'applicazione diretta di questa topologia basilare a filtri del secondo ordine introduce asimmetrie e alterazioni di fase indesiderate nei pressi della frequenza di taglio. Nella realtà (e nel progetto) si usano approcci differenti [3, 14].

In JUCE sono istanziati con:

```
coeffs = juce::dsp::IIR::Coefficients<float>::makeLowShelf(
    sampleRate, fc, Q, g);
coeffs = juce::dsp::IIR::Coefficients<float>::makeHighShelf(
    sampleRate, fc, Q, g);
```

Passando al modulo dedicato all'analisi, il progetto utilizza i filtri passa-banda (band-pass) per isolare l'energia delle 8 ottave centrali (al fine di estrarne il valore RMS). Il suo prototipo analogico a due poli è definito dall'equazione $H(s) = \frac{s}{s^2 + 2Rs + 1}$ [16]. L'obiettivo di questo filtro è avere un guadagno unitario e uno sfasamento nullo alla frequenza centrale (di interesse), provocando contemporaneamente un'attenuazione progressiva delle frequenze esterne [16], come visibile in Figura 11.

Figura 11: Risposta del filtro band-pass al variare di R [16].

In JUCE istanziato con:

```
coeffs = juce::dsp::IIR::Coefficients<float>::makeBandPass(
    sampleRate, fc, Q);
```

Infine, vengono usati filtri passa-basso e passa-alto (low-pass e high-pass) per isolare l'analisi delle due bande estreme. I prototipi a due poli alla base di questi filtri sono modellizzati dalle funzioni $H(s) = \frac{1}{s^2+2Rs+1}$ per il passa-basso e $H(s) = \frac{s^2}{s^2+2Rs+1}$ per il passa-alto [16] (Figura 12).

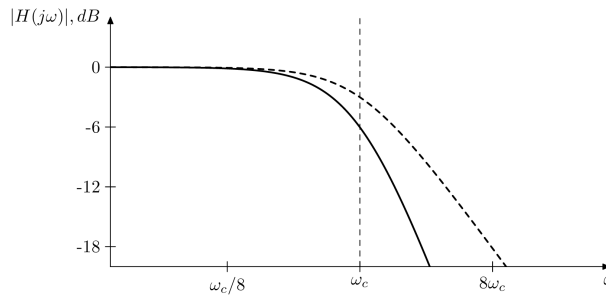


Figura 12: Risposta del filtro passa-basso a due poli [16].

In JUCE istanziati con:

```
coeffs = juce::dsp::IIR::Coefficients<float>::makeLowPass(
    sampleRate, fc, Q);
coeffs = juce::dsp::IIR::Coefficients<float>::makeHighPass(
    sampleRate, fc, Q);
```

Capitolo 4

Progettazione e Implementazione del Software

4.1 Organizzazione architetturale e separazione logica

L'architettura del plug-in è stata ideata applicando una rigorosa separazione dei compiti per garantire modularità al codice, facilità di manutenzione e nell'estensione. Il diagramma a blocchi in Figura 13 illustra il flusso dei dati e le interazioni tra i moduli (e file) principali.

Per mantenere snello il modulo di processing, gran parte del template JUCE è stato delegato ad una classe genitrice dedicata, denominata `AbstractAudioProcessor`¹, da cui `GraphicEQAudioProcessor`² eredita. Questa scelta permette di lasciare il processore principale focalizzato esclusivamente sulle operazioni DSP.

La definizione e la gestione dei parametri accessibili all'utente (i 10 fader di equalizzazione, il guadagno globale e il selettore di routing per il matching) sono demandate all'oggetto `AudioProcessorValueTreeState` (VTS), strutturato all'interno del file `PluginParameters.h`³. Il VTS funge da "ponte di comunicazione" del plug-in poiché garantisce la sincronizzazione thread-safe bidirezionale tra l'interfaccia grafica (`PluginEditor`⁴) e il motore audio (`PluginProcessor`⁵).

¹Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/AbstractAudioProcessor.h>

²Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginProcessor.h>

³Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginParameters.h>

⁴Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginEditor.h>

⁵Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginProcessor.cpp>

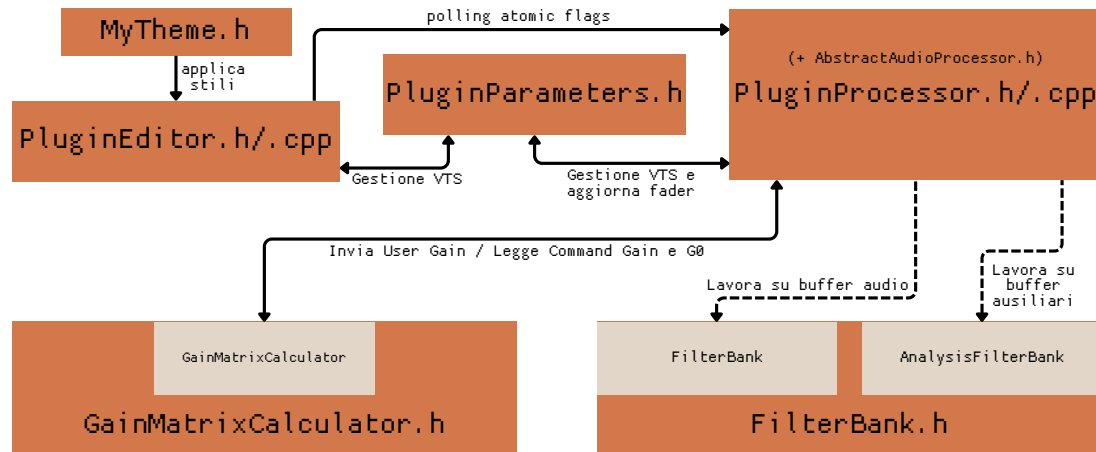


Figura 13: Diagramma architetturale e flusso dei dati tra le classi del plug-in.

In questa architettura, come si vede nello schema logico, il `PluginProcessor` agisce da controller. Non esegue direttamente i calcoli matematici per la matrice di interazione, né l'applicazione diretta dei filtri ai buffer audio, ma delega queste mansioni a tre moduli specializzati indipendenti: `GainMatrixCalculator`, `FilterBank` e `AnalysisFilterBank`.

4.1.1 `GainMatrixCalculator`: il modulo matematico

La classe `GainMatrixCalculator`⁶ costituisce il modulo matematico del plug-in. Al suo interno sono centralizzate e inizializzate hard-coded (sotto forma di look-up table) le costanti fondamentali del progetto, ossia l'elenco delle dieci frequenze centrali ISO e i coefficienti della matrice di interazione. Le frequenze sono rese disponibili agli altri moduli tramite apposite funzioni getter (come `const float* getCenterFrequencies() const`), evitando qualsiasi duplicazione di dati nell'architettura e mantenendo tutto coerente (verranno usate per inizializzare filtri, nella GUI e così via).

La parte fondamentale della classe consiste nel seguente metodo:

```
void update(const float* userGain);
```

⁶Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/GainMatrixCalculator.h>

Ogni volta che avviene una variazione grafica sull'interfaccia, infatti, il processore passa a questa funzione l'array dei guadagni impostati dall'utente (gli user gain). Avvalendosi delle strutture dati fornite dalla libreria Eigen, il metodo esegue matematicamente i passaggi teorici del disaccoppiamento:

- Estrazione dell'offset: calcola la media aritmetica del vettore degli user gain, ricavando il guadagno globale G_0 .
- Isolamento dell'involuppo: sottrae G_0 dal vettore originale, ottenendo i guadagni al netto della variazione di volume.
- Calcolo matriciale: moltiplica il nuovo vettore per la matrice di interazione, calcolando istantaneamente i command gain definitivi.

I risultati di queste operazioni vengono immagazzinati nelle variabili membro omonime della classe e possono essere interrogati in modo sicuro dal processore tramite i seguenti metodi, pronti per essere applicati ai filtri e allo stadio di guadagno finale:

```
const float* getCommandGain() const;  
float getGlobalOffset() const;
```

4.1.2 FilterBank: applicazione dei filtri di equalizzazione

La classe `FilterBank`⁷ rappresenta il modulo DSP che si occupa dell'elaborazione audio effettiva sul percorso del segnale principale. Il costruttore della classe alloca dinamicamente una struttura dati contenente 20 filtri (10 per canale) di tipo Infinite Impulse Response (IIR), suddivisi nelle 10 bande di frequenza ISO.

Prima del processamento vero e proprio, la funzione preparatoria del processore invoca il metodo di questa classe:

```
void prepareToPlay(double sr, const float* fc);
```

Questa funzione serve per inizializzare il banco filtri: riceve la frequenza di campionamento, associa in modo permanente le dieci frequenze centrali ISO alle rispettive bande e forza un `reset()` per ripulire le memorie di ritardo dei filtri.

Il nucleo della gestione dei filtri risiede nel metodo privato `updateCoefficients()`, incaricato di tradurre i valori di guadagno in coefficienti biquadratici tramite il modulo `juce::dsp`. La topologia dei filtri generata da questa funzione rispecchia l'architettura per equalizzatori grafici formalizzata da Välimäki [3], utilizzando filtri shelving agli estremi e peaking al centro, con un fattore di merito costante pari a $Q = \sqrt{2}$. Il ricalcolo dei coefficienti viene innescato quando necessario: durante l'inizializzazione e ogni volta che viene chiamato il metodo pubblico:

⁷Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/FilterBank.h>

```
void setGains(const float* newCommandGain);
```

L’elaborazione in tempo reale avviene infine nel seguente metodo. Ricevendo il buffer dal processore principale, la funzione itera sulle bande e sui canali applicando il filtraggio in cascata:

```
void processBlock(AudioBuffer<float>& buffer, int numSamples, int numCh)
    ;
```

4.1.3 AnalysisFilterBank: analisi energetica delle bande frequenziali

La classe `AnalysisFilterBank`⁸ costituisce il modulo DSP dedicato esclusivamente all’analisi dei segnali target e reference. Il costruttore della classe alloca dinamicamente una struttura dati contenente 10 filtri IIR, uno per ciascuna banda ISO, utilizzati per isolare le bande del caso in fase di analisi.

Come per il modulo precedente, prima che avvenga qualsiasi calcolo il processore principale invoca l’omonimo metodo di questa classe:

```
void prepareToPlay(double sr, const float* fc);
```

Questa funzione inizializza il banco filtri ricevendo la frequenza di campionamento, associa le frequenze centrali ISO alle bande, forza un `reset()` per azzerare gli stati interni e innesca il calcolo iniziale dei coefficienti.

Il metodo privato `updateCoefficients()` definisce una topologia dei filtri specifica per l’analisi, configurati come passa-basso (low-pass) per la prima banda, passa-alto (high-pass) per la decima, e passa-banda (band-pass) per le otto frequenze centrali. Il fattore di merito viene impostato a un valore fisso restrittivo, pari a $Q = 10$, ricavato sperimentalmente. Poiché questi filtri servono solo come “finestre” di ascolto, i loro coefficienti non vengono più ricalcolati dopo l’inizializzazione.

L’estrazione dei dati avviene poi nel metodo pubblico:

```
void analyze(const juce::AudioBuffer<float>& buffer, std::array<float,
    MAX_NUM_BANDS>& analysis);
```

Ricevendo in ingresso il buffer ausiliario (catturato durante la fase di apprendimento), la funzione itera sulle 10 bande eseguendo una sequenza precisa: effettua una copia temporanea del segnale, applica il filtro IIR della banda corrente e, infine, utilizza la funzione `getRMSLevel()` nativa di JUCE per calcolare l’energia quadratica media del segnale isolato. Il valore RMS ottenuto viene salvato nell’apposito array passato per riferimento dall’algoritmo di matching.

⁸Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/FilterBank.h>

4.2 Modulo di equalizzazione grafica

4.2.1 Il percorso del segnale audio

L'elaborazione del segnale avviene nel metodo `processBlock`⁹ della classe processore ma, come anticipato nell'analisi architetturale, questa funzione è stata mantenuta volutamente snella per operare unicamente come “direttore d'orchestra” del flusso audio:

```
void GraphicEQAudioProcessor::processBlock (juce::AudioBuffer<float>&
    buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;

    auto& targetBuffer = getBusBuffer(buffer, true, 0);
    auto& sidechainBuffer = getBusBuffer(buffer, true, 1);

    if (learningTarget.load() || learningReference.load())
        handleLearn(targetBuffer, sidechainBuffer);

    const auto numSamples = targetBuffer.getNumSamples();
    const auto numCh = targetBuffer.getNumChannels();

    filters.processBlock(targetBuffer, numSamples, numCh);

    totalGainSmoothed.applyGain(targetBuffer, numSamples);
}
```

Come si può vedere dal codice, ci sono tre fasi sequenziali. Inizialmente si ha la scomposizione del buffer principale nei rispettivi bus di ingresso (main e sidechain). Successivamente, un blocco condizionale verifica lo stato di due flag atomici: se l'utente ha avviato una fase di acquisizione (learn), i buffer vengono passati alla funzione `handleLearn` che si occuperà di registrare l'audio in memorie ausiliarie, lasciando il segnale originale inalterato per proseguire il suo percorso. Infine, avviene il processamento timbrico vero e proprio sul segnale principale (target). Il compito del filtraggio è interamente delegato all'istanza `filters` (ossia la classe `FilterBank` precedentemente descritta), dopodiché al segnale risultante viene applicato il guadagno globale.

Si può notare come tutta la gestione dei parametri dei filtri, degli eventi, e dei calcoli non avvenga qui. Saranno infatti i moduli del caso ad occuparsene.

⁹Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginProcessor.cpp>

4.2.2 Gestione dei parametri e aggiornamento dei filtri

La gestione “dietro le quinte” delle variabili matematiche avviene all’interno del metodo `parameterChanged`¹⁰. Questa funzione viene richiamata in modo automatico ogni volta che il `ValueTreeState` rileva un’interazione dell’utente con i controlli dell’interfaccia grafica.

È in questo blocco che si concretizza l’interazione tra i moduli logici del plug-in. Quando viene rilevata la modifica di un fader di equalizzazione, viene eseguita la seguente catena di chiamate:

```
// 1. Aggiorna lo stato interno degli userGain impostati dall'utente
userGain[band] = newValue;

// 2. Aggiorna i commandGain e calcola l'offset G0 tramite
    GainMatrixCalculator
gmc.update(userGain);

// 3. Aggiorna i coefficienti dei filtri tramite FilterBank
filters.setGains(gmc.getCommandGain());

// 4. Aggiorna il volume
updateTotalGainTarget();
```

Come prima cosa, il processore aggiorna l’array interno dei guadagni utente (`userGain`) e lo trasmette all’oggetto `gmc` (istanza della classe `GainMatrixCalculator`). Quest’ultimo esegue le operazioni matriciali, calcolando i command gain e G_0 . Questi vengono immediatamente inoltrati a `FilterBank` per il ricalcolo dei coefficienti dei filtri. Infine si chiama il metodo ausiliario `updateTotalGainTarget()`, che si occupa di gestire il volume complessivo in uscita:

```
void GraphicEQAudioProcessor::updateTotalGainTarget() {
    auto G0 = gmc.getGlobalOffset();
    auto totalGainDb = G0 + globalGain;

    auto linearGain = juce::Decibels::decibelsToGain(totalGainDb);
    totalGainSmoothed.setTargetValue(linearGain);
}
```

Come si vede dal codice, il volume totale da applicare al segnale non dipende unicamente dal parametro `globalGain` (il controllo di volume mosso fisicamente dall’utente), ma richiede il reinserimento di G_0 . Sommando l’offset estratto dalla matrice al volume globale, si garantisce che l’energia rimossa per ottimizzare la risposta dei filtri venga reintrodotta. Il risultato in decibel viene infine convertito in scala lineare e assegnato come

¹⁰Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginProcessor.cpp>

target a `totalGainSmoothed`, che provvederà ad applicarlo in modo continuo e smussato all'interno della `processBlock`.

4.3 Modulo di matching

Il modulo del matching è concepito per analizzare il contenuto spettrale di due segnali distinti e adattare automaticamente la curva dell'equalizzatore. Per non interferire con le prestazioni real-time, questo processo è stato costruito per operare interamente offline (e anche su thread separato) sui dati precedentemente catturati, dividendosi in una fase di acquisizione (`learn`) e in una fase di analisi (`match`).

4.3.1 Fase di learn

La pressione dei pulsanti “Learn Target” e “Learn Reference” sulla GUI¹¹ innesca, tramite l'evento `onClick`, la chiamata dei metodi trigger nel processor: `triggerLearnTarget()` e `triggerLearnReference()`.

Questi trigger verificano innanzitutto che non sia già in corso un'acquisizione e invocano in caso negativo delle specifiche funzioni preparatorie (`startLearnTarget()` e `startLearnReference()`). In queste, il sistema viene resettato: i buffer ausiliari vengono svuotati, i contatori dei campioni azzerati e i flag di stato (usati per gestire tutto il processo di `learn`) aggiornati.

Tutta la gestione del flusso di `learn` è gestita tramite l'utilizzo di variabili `std::atomic<bool>` (come `learningTarget`, `learningReference`, `hasLearnedTarget`, `hasLearnedReference`). Tutte le funzioni coinvolte infatti leggono o modificano i loro valori per gestire l'inizio e la fine del processo.

Come mostrato prima nell'analisi della `processBlock`, il processore verifica ad ogni ciclo il valore di questi flag atomici. Se risulta attiva una fase di learning, i buffer di ingresso vengono passati al metodo dedicato:

```
void handleLearn(const juce::AudioBuffer<float>& inputBuffer, const juce::AudioBuffer<float>& sidechainBuffer);
```

Questa funzione si occupa di riempire i buffer ausiliari (sia target che reference) in modo indipendente, operando in base allo stato dei vari flag di comando.

Questi buffer vengono allocati in fase di inizializzazione del plugin e rimangono in memoria per tutta la vita di questo, e sono strutturati per contenere esattamente 5 secondi di audio in formato mono. Per questo motivo, nel caso in cui il segnale in ingresso si presenti stereofonico, la funzione applica un `downmix` durante la copia dei buffer. Inoltre, la `handleLearn` gestisce il routing del segnale di reference, prelevando l'audio dal bus

¹¹Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginEditor.cpp>

di sidechain se l'apposita opzione è abilitata, oppure effettuando un fallback automatico sull'ingresso principale.

Il riempimento dei buffer procede in modo incrementale. Non appena il contatore dei campioni raggiunge la capienza massima prefissata, la logica interna interrompe autonomamente l'acquisizione e aggiorna lo stato dei flag. Questo aggiornamento funge da "semaforo verde": da un lato segnala all'interfaccia grafica di ripristinare e sbloccare i controlli visivi, dall'altro abilita a livello logico l'esecuzione del match, interrompendo allo stesso tempo la fase di learn.

4.3.2 Fase di match

Una volta completata l'acquisizione, la pressione del pulsante "Match!" sulla GUI innescava la fase di match tramite il metodo `triggerMatch()`, che al suo interno richiama la funzione `handleMatch()`. Dato che l'analisi potrebbe richiedere una notevole potenza di calcolo al punto da compromettere la stabilità dell'audio real-time, questa funzione viene chiamata utilizzando una chiamata asincrona: `juce::MessageManager::callAsync`. In questo modo, il controllo passa in modo "indipendente" al metodo `handleMatch()`, che funge da coordinatore dell'intero processo.

Come prima operazione, questo metodo crea delle copie temporanee dei buffer ausiliari, al fine di preservare i dati originali. Su queste copie viene invocata la funzione:

```
void normalizeBuffer(juce::AudioBuffer<float>& buffer);
```

la quale calcola il livello RMS globale del segnale e vi applica un guadagno inversamente proporzionale. Questa fase è fondamentale per pareggiare i volumi dei segnali target e reference prima di poterne confrontare coerentemente il timbro. Terminata la normalizzazione, il processore sfrutta l'istanza `analysisFilters` (oggetto della classe `AnalysisFilterBank` precedentemente descritta) per estrarre le energie RMS di ciascuna delle 10 bande, memorizzando i risultati in due array distinti.

I dati spettrali appena estratti vengono quindi passati alla seguente funzione iterativa, la quale calcola matematicamente il guadagno necessario affinché il target replichi la reference:

```
void generateEqCurve(const std::array<float, MAX_NUM_BANDS>&
    targetAnalysis,
    const std::array<float, MAX_NUM_BANDS>&
        referenceAnalysis,
    std::array<float, MAX_NUM_BANDS>& eqCurveDb);
```

L'ultimo passaggio consiste nell'applicare concretamente i guadagni calcolati al plugin. Iterando sulle 10 bande, il processore recupera il riferimento a ciascun parametro dal `ValueTreeState` e ne aggiorna il valore richiamando l'istruzione:

`floatParam->setValueNotifyingHost(...)`. Questa chiamata produce un effetto a cascata: da un lato muove fisicamente i fader sull'interfaccia grafica (fornendo un

feedback visivo immediato all'utente), dall'altro innesca automaticamente la funzione `parameterChanged` esaminata nel paragrafo precedente. Il sistema procede così autonomamente al ricalcolo degli `user gain`, `command gain`, G_0 e al definitivo aggiornamento dei coefficienti nel `FilterBank`.

4.4 Interfaccia grafica

L'estetica generale e la personalizzazione dei componenti grafici (colori, forme e font) sono gestite tramite una classe dedicata, `MyTheme`¹², che estende e sovrascrive i comportamenti del `LookAndFeel` nativo di JUCE. Tutto il resto della GUI è gestita da `PluginEditor.cpp/.h`¹³.

La finestra principale (Figura 14) espone tutti i controlli disponibili. La sezione di equalizzazione è composta da dieci fader verticali, ciascuno con un'escursione di ± 12 dB e corredato da un box di testo inferiore che mostra l'esatto valore di guadagno della banda. Un elemento distintivo del design è la linea continua che unisce i cursori dei fader: non viene disegnata la risposta in frequenza precisa dei filtri sovrapposti, ma offre una rappresentazione visiva creativa ed elegante dell'involuppo, realizzata avvalendosi dei path grafici di JUCE smussati tramite l'interpolazione di curve cubiche.

In basso, si ha un fader per il volume generale, un interruttore (toggle) per abilitare il bus di `sidechain` come sorgente `reference`, e i tre pulsanti operativi "Learn Target", "Match!" e "Learn Reference".

L'interfaccia è inoltre dotata di un comportamento dinamico: durante le fasi di `learn`, l'interfaccia si adatta bloccando i controlli "occupati". Come illustrato nelle Figure 15a e 15b, l'avvio del `Learn Target` disabilita temporaneamente il proprio pulsante per impedire pressioni sovrapposte. Analogamente, l'avvio del `Learn Reference` blocca il pulsante relativo e, contestualmente, lo stato del selettore `sidechain`, impedendo che l'utente cambi il routing del segnale a registrazione in corso.

Infine, il pulsante centrale di "Match!" si disabilita automaticamente durante qualsiasi fase di apprendimento e "non fa nulla" finché il sistema non completa il `learn` di entrambi il `target` e `reference`.

¹²Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/MyTheme.h>

¹³Codice sorgente disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/blob/main/PluginEditor.h>

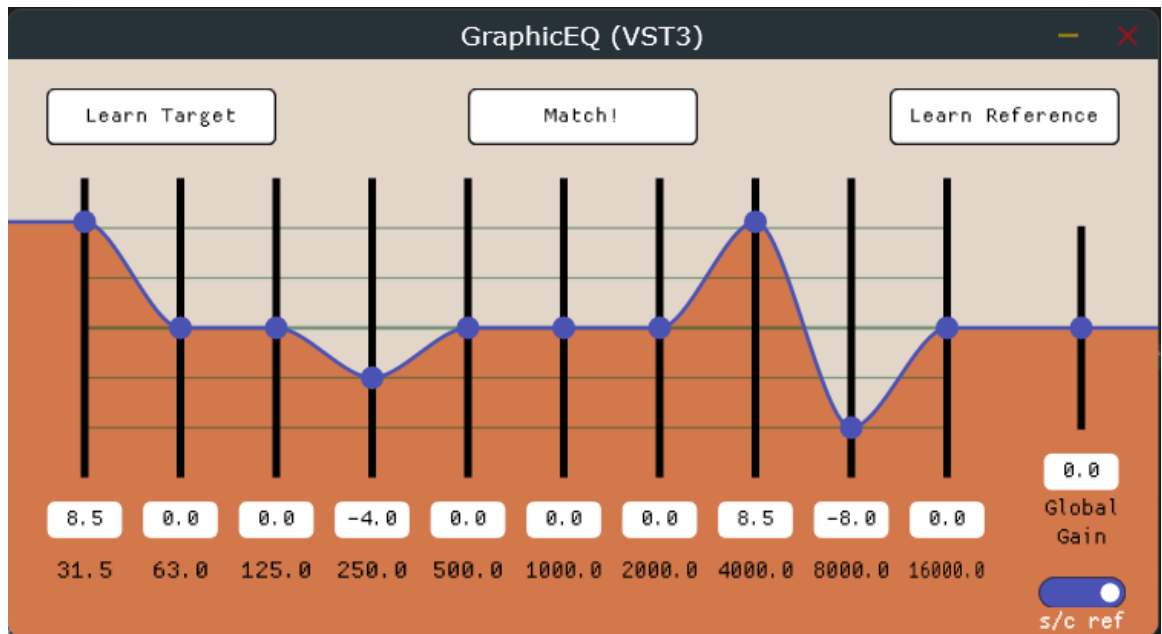
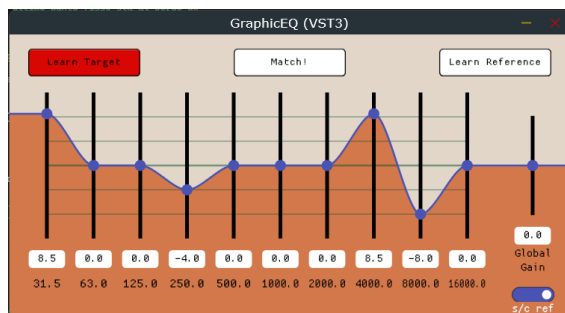
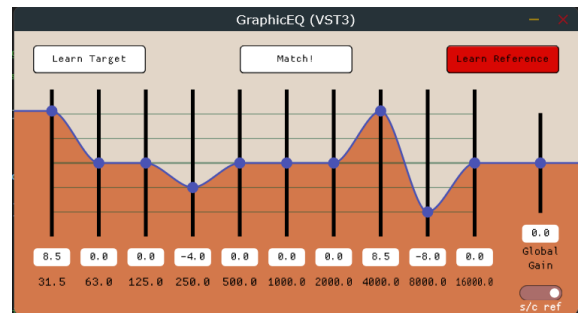


Figura 14: Schermata principale dell'interfaccia utente.



(a) Disabilitazione durante Learn Target.



(b) Disabilitazione durante Learn Reference.

Figura 15: Comportamento dinamico dell'interfaccia e disabilitazione dei controlli durante le fasi di apprendimento (learn).

Capitolo 5

Test e Validazione

5.1 Metodologia e strumenti di misurazione spettrale

Per valutare le prestazioni del plug-in, in questo capitolo si espone una fase di testing strutturata in due parti distinte: la validazione del meccanismo di compensazione e la misurazione dell'accuratezza dell'algoritmo di *matching*. L'approccio inoltre separa l'acquisizione dei segnali audio dalla successiva analisi dei dati.

L'intera fase di acquisizione audio è avvenuta all'interno della Digital Audio Workstation (DAW) Ableton Live. Questo ambiente *host* è stato usato per istanziare il plug-in, generare ed elaborare i segnali di test, ed esportare le tracce processate in formato audio non compresso.

La successiva valutazione matematica è stata svolta sviluppando script dedicati in linguaggio Python, avvalendosi delle librerie scientifiche NumPy, SciPy [17] e Matplotlib per la renderizzazione grafica. Nello specifico, gli strumenti di misurazione adottati per le due fasi di test (e contenuti dentro le librerie) sono i seguenti:

Metodo di Welch (fase di validazione): tramite il metodo di Welch [18], è stato implementato il calcolo della Power Spectral Density (PSD) per ricavare la risposta in frequenza dei segnali. Questo algoritmo segmenta il segnale in finestre temporali sovrapposte ed estrae la media dei periodogrammi. Questo riduce la varianza statistica del segnale, garantendo curve spettrali precise.

Log-Spectral Distance (fase di valutazione Matching): poi per quantificare l'accuratezza del matching, l'estrazione spettrale di Welch è stata combinata con il calcolo della Log-Spectral Distance (LSD) [19]. Questa metrica standard fornisce una stima numerica dell'errore quadratico medio tra due spettri, permettendo di misurare la convergenza tra un segnale di riferimento (reference) e l'equalizzazione prodotta autonomamente dal plug-in.

5.2 Validazione della matrice di interazione: coerenza tra user gain e risposta in frequenza effettiva

Lo scopo di questa fase di validazione è verificare l'accuratezza del modello matematico basato sul metodo proposto da Abel e implementato nella classe GainMatrixCalculator. Il test mira a dimostrare che i command gain generati dalla matrice restituiscono una curva di densità spettrale coincidente con l'interfaccia grafica (o meglio, con i valori user gain settati dall'utente).

Per il testing, l'equalizzatore è stato sottoposto a cinque ipotetici scenari di utilizzo, partendo da casistiche standard fino ad arrivare a *stress test*. Tutto applicato a del rumore bianco.

Test 1: esaltazione di una singola banda

In questo test si interviene su singolo filtro peaking (impostato a +12 dB sulla frequenza di 1 kHz) mantenendo il resto dello spettro inalterato (0 dB). Il test permette di verificare il corretto centramento delle frequenze e l'assenza di anomalie nel guadagno di base (Figura 16).

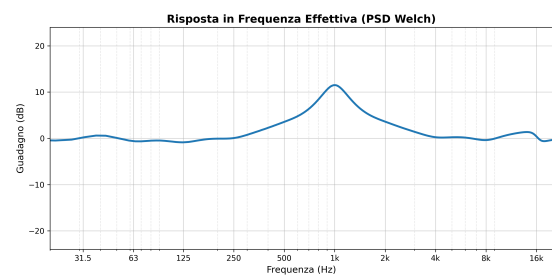
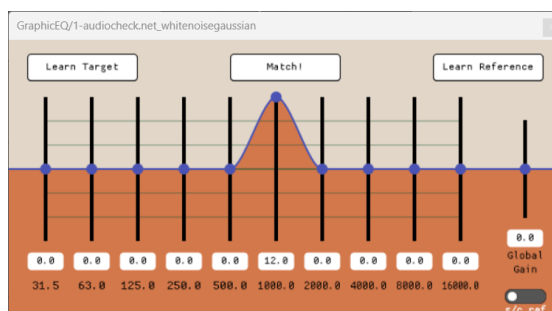


Figura 16: Test 1 - configurazione esaltazione singola banda a 1 kHz

Analisi del risultato: Il grafico conferma un tracciamento perfetto, dove si ha il picco risonante che raggiunge esattamente i +12 dB in corrispondenza di 1 kHz, mentre le parti di spettro non interessate dalla campana del filtro si mantengono stabili sulla linea dello zero.

Test 2: curva di equalizzazione standard (*smile curve*)

Questo test emula un tipico scenario di utilizzo musicale, dove si ha un'esaltazione delle basse e delle alte frequenze (verso i +8 dB) accompagnata da uno svuotamento delle medie (verso i -8 dB). L'obiettivo è verificare la transizione morbida tra i filtri (Figura 17).

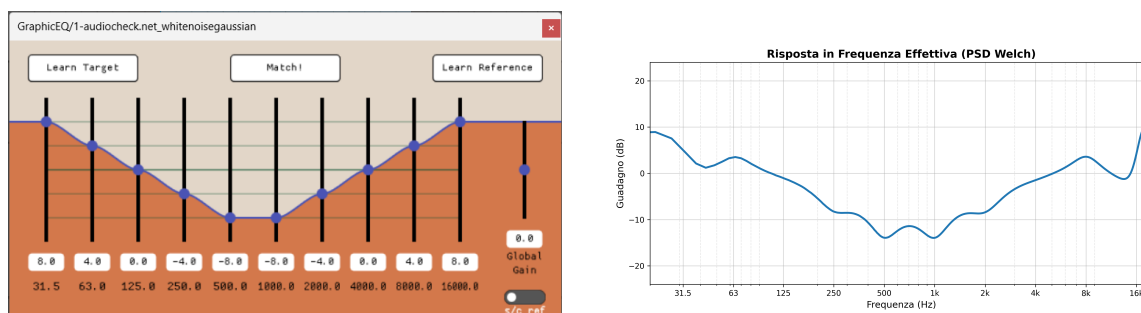


Figura 17: Test 2 - configurazione curva di equalizzazione standard

Analisi del risultato: L'involuppo generato rispecchia in modo fedele il tracciato dei fader. Le regioni esaltate raggiungono i target previsti (+8 dB), mentre l'attenuazione centrale si attesta intorno ai -10 dB.

Test 3: stress test a bande alternate

Questo rappresenta il test di sforzo massimo per la matrice di interazione. I fader sono posizionati alternando il massimo guadagno (+12 dB) alla massima attenuazione (-12 dB). In un equalizzatore senza meccanismi di compensazione questo genererebbe una risposta caotica, qui si vuole testare il limite di efficacia del calcolo matriciale (Figura 18).

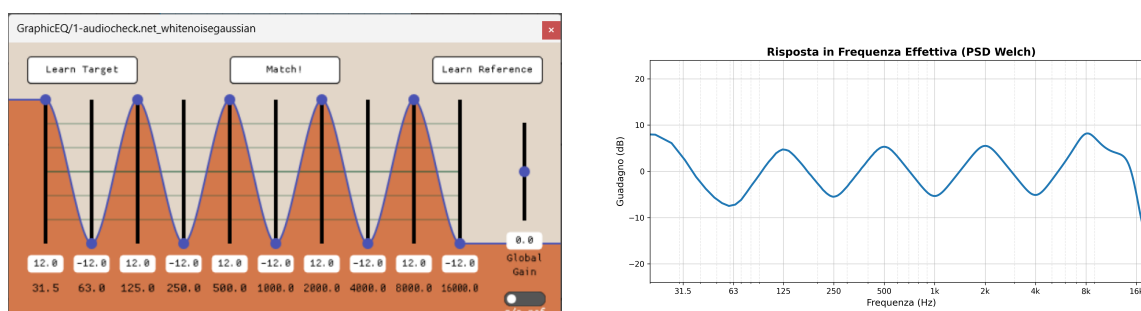


Figura 18: Test 3 - configurazione stress test bande alternate

Analisi del risultato: L'andamento acustico rispecchia in modo accurato il pattern visivo a "zig-zag", tuttavia si può vedere che i picchi non raggiungono i valori massimi voluti di ± 12 dB, ma si limitano ad escursioni più contenute. Questo comportamento è in realtà aspettato, dato che per costringere il segnale a toccare i ± 12 dB in bande separate da una sola ottava la matrice dovrebbe calcolare dei command gain interni con valori molto alti (es. ± 40 dB), e questo causerebbe inevitabilmente saturazione del DSP e gravi distorsioni di fase. Quindi il codice calcola "un compromesso" mantenendo la proporzione della curva richiesta dall'utente ma garantendo stabilità e limiti del segnale audio.

Test 4: emulazione di un high-pass

Questo test cerca di verificare l'efficienza della combinazione di filtri per simulare un taglio netto sulle basse frequenze (stile passa-alto), lasciando intatta la metà superiore dello spettro (Figura 19).

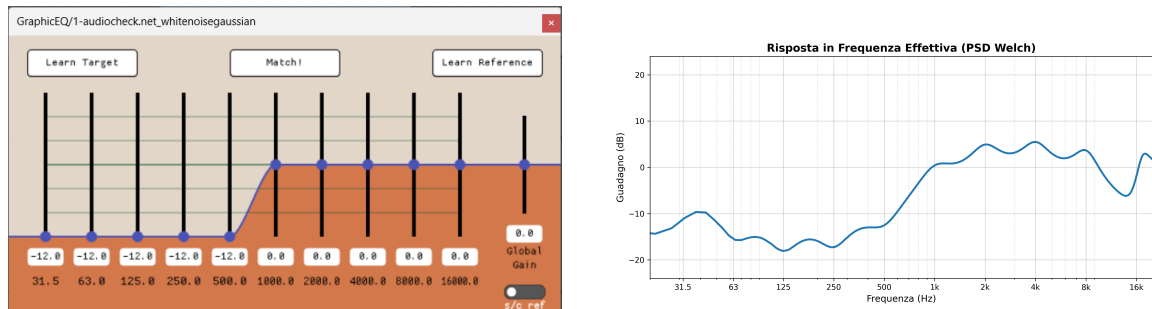


Figura 19: Test 4 - configurazione emulazione di un high-pass

Analisi del risultato: La riduzione sulle basse frequenze è coerente con l'interfaccia, e la transizione verso la metà superiore dello spettro (impostata a 0 dB) risulta fedele alla curva voluta. Le oscillazioni visibili sono tollerabili e dovute appunto all'uso di filtri IIR del secondo ordine in cascata.

Test 5: configurazione casuale asimmetrica

Quest'ultimo test ricrea un comportamento imprevedibile, dove si posizionano i fader in modo asimmetrico e casuale, per controllare ulteriormente la coerenza su curve non convenzionali (Figura 20).

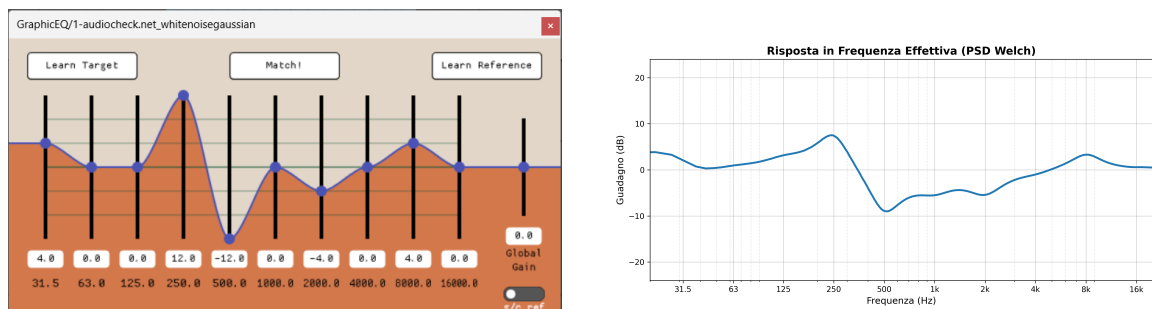


Figura 20: Test 5 - configurazione asimmetrica casuale

Analisi del risultato: Anche in questo scenario l'involuppo segue in modo abbastanza accurato il profilo tracciato dai cursori. Analogamente a quanto osservato nel test 3, nelle zone in cui si verificano conflitti elevati tra fader adiacenti (salti bruschi di guadagno), la matrice interviene limitando l'escursione di picco.

5.3 Valutazione dell'algoritmo di matching: misurazione della convergenza spettrale tra target e reference

Quest'ultima fase di testing ha lo scopo di valutare la fase di matching del plug-in, quindi la capacità del modulo learn/match di estrarre l'involuppo spettrale di un segnale di riferimento (la reference) e di calcolare automaticamente la curva di equalizzazione ottimale dei 10 fader per approssimare tale curva sul segnale in ingresso.

Per l'analisi è stata usata una metrica standard: la Log-Spectral Distance (LSD) [19]. Questa misura esprime (in decibel) l'errore quadratico medio tra due spettri di potenza espressi in scala logaritmica. Questa metrica viene applicata ai dati estratti tramite il metodo di Welch [18], e viene definita come:

$$LSD = \sqrt{\frac{1}{K} \sum_{k=1}^K (10 \log_{10} |P_{ref}(k)| - 10 \log_{10} |P_{match}(k)|)^2}$$

dove K rappresenta il numero di bin frequenziali validi all'interno dello spettro udibile (20 Hz - 20 kHz). Un valore di LSD intorno allo zero indica una sovrapposizione perfetta tra i due segnali.

Per il test sono stati preparati tre scenari differenti. Per ciascuno, un segnale audio "pulito" è stato processato in Ableton Live tramite un equalizzatore parametrico (EQ Eight, stock di Ableton) per generare la reference. Successivamente, il plug-in sviluppato ha analizzato questa reference e creato una curva di equalizzazione matching.

Nei grafici che seguono si vedono analizzati e comparati i segnali originali (puliti), la reference, e il segnale target elaborato dal plug-in. Viene infine calcolato il valore LSD.

Test 1: rumore bianco

In questo test, un segnale di rumore bianco è stato sottoposto a una forte equalizzazione parametrica (utilizzando l'EQ Eight stock di Ableton), configurata per creare transizioni brusche e strette sullo spettro. La reference è stata generata utilizzando quattro filtri: filtro low-shelf a 60 Hz (+8 dB, $Q = 1$), filtro a campana a 500 Hz (-8 dB, $Q = 4$), filtro a campana a 4 kHz (+8 dB, $Q = 4$), filtro high-shelf a 10 kHz (-8 dB, $Q = 0.71$).

Per fornire un riscontro visivo, di seguito vengono affiancati l'equalizzatore di riferimento (EQ Eight, le cui modifiche "creano" di fatto la reference) e l'interfaccia del plug-in, che mostra come l'algoritmo ha posizionato autonomamente i propri fader per imitarne la curva dopo la fase di learn/match (Figura 21).

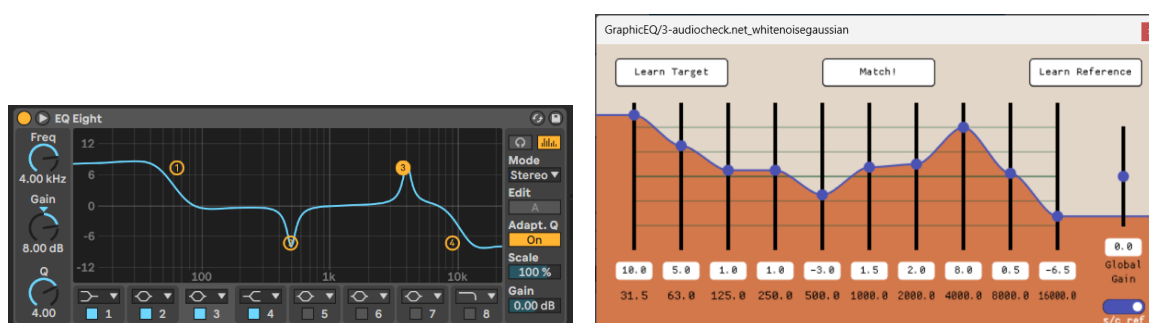


Figura 21: Test 1 - confronto interfacce su rumore bianco

Poi, per un'analisi spettrale accurata, viene mostrato il grafico con le densità di potenza a confronto (Figura 22), includendo il segnale pulito originale (linea grigia a puntini), la curva della reference (arancione) e il segnale effettivamente restituito dal plug-in (blu tratteggiato).

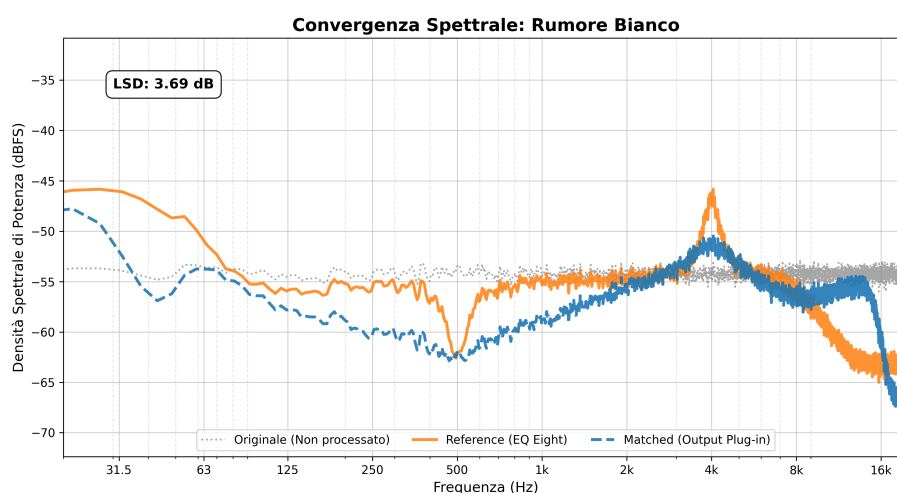


Figura 22: Test 1 - grafico densità di potenza su rumore bianco

Analisi del risultato: L'algoritmo ha tracciato l'involuppo con una buona precisione, ottenendo un LSD di 3.69 dB. Come si vede dal grafico, il plug-in (curva tratteggiata blu) riesce a ricreare l'alternanza di picchi e valli imposta dalla reference. Il sistema insegue sia l'innalzamento sulle basse frequenze sia il taglio a 500 Hz. La lieve divergenza visibile attorno alle altissime frequenze è dovuta all'approssimazione delle campane fisse ISO rispetto all'equalizzazione chirurgica utilizzata come riferimento.

Test 2: brano pop

Questo test impiega un segnale musicale denso e moderno, nello specifico il brano "Redecorate" dei Twenty One Pilots. La reference è stata creata imponendo tre filtri: filtro

a campana a 60 Hz (+8 dB, $Q = 2.5$) (*boost* stretto sui *sub-bass*), filtro a campana a 1500 Hz (-8 dB, $Q = 3$), filtro high-shelf a 8000 Hz (+6 dB, $Q = 0.71$).

Di seguito si riportano l'interfaccia dell'equalizzatore di riferimento e la configurazione dei fader elaborata dal plug-in in fase di matching (Figura 23).

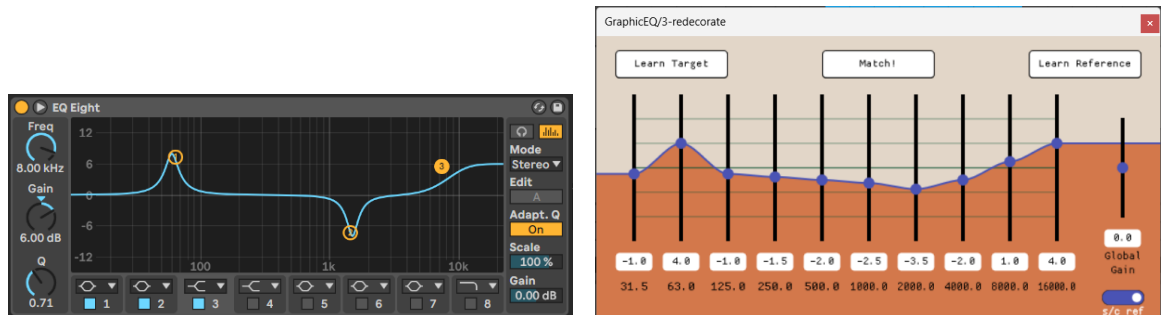


Figura 23: Test 2 - confronto interfacce su brano pop

L'analisi spettrale accurata delle tre curve è riportata nel grafico seguente (Figura 24).

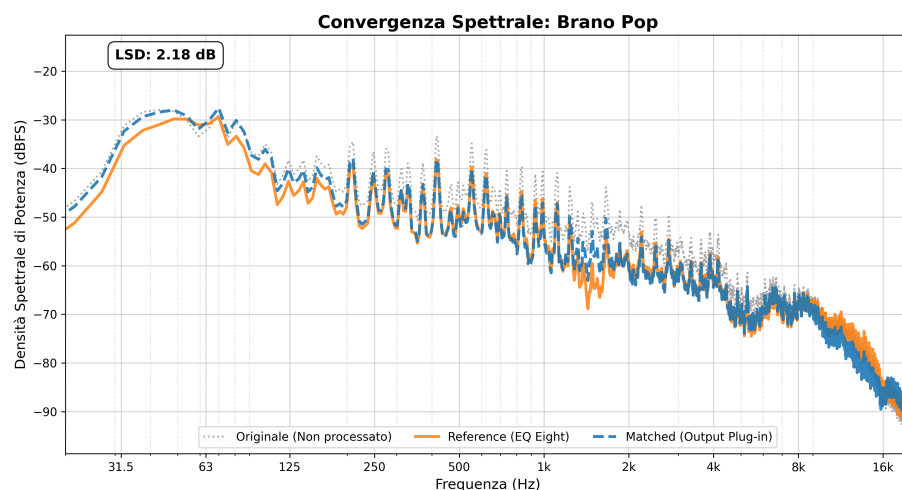


Figura 24: Test 2 - grafico densità di potenza su brano pop

Analisi del risultato: con un LSD di soli 2.18 dB, il livello di sovrapposizione visibile nel grafico è molto buono. Nonostante i centri banda non corrispondessero esattamente alle frequenze alterate dalla reference, il plug-in ha mediato correttamente l'energia.

Test 3: brano rock

In questo test, che utilizza il brano rock “Inospitale” dei Gazebo Penguins, è stato applicato un marcato effetto “Lo-Fi” per privare la traccia dei suoi estremi di banda, usando: filtro passa-alto a 150 Hz ($Q = 0.71$), filtro a campana a 500 Hz (+8 dB, $Q = 1$), filtro passa-basso a 6000 Hz ($Q = 0.71$).

Di seguito si riportano l’interfaccia dell’equalizzatore di riferimento e la configurazione dei fader elaborata dal plug-in in fase di matching (Figura 25).

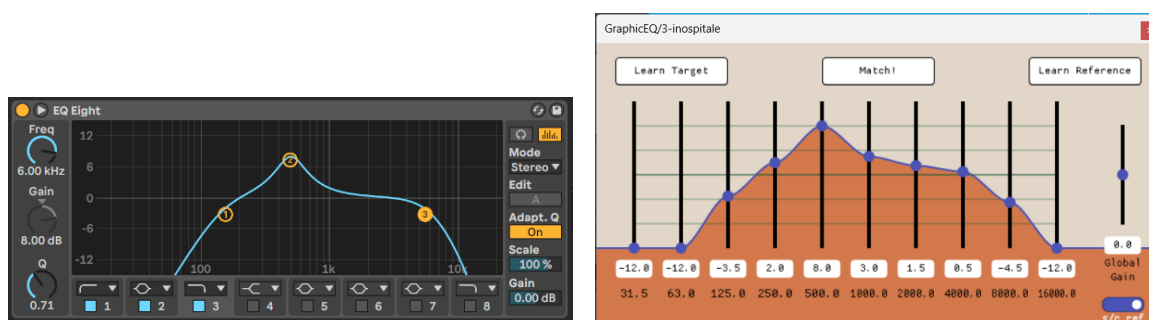


Figura 25: Test 3 - confronto interfacce su brano rock

L’analisi spettrale accurata delle tre curve è riportata nel grafico seguente (Figura 26).

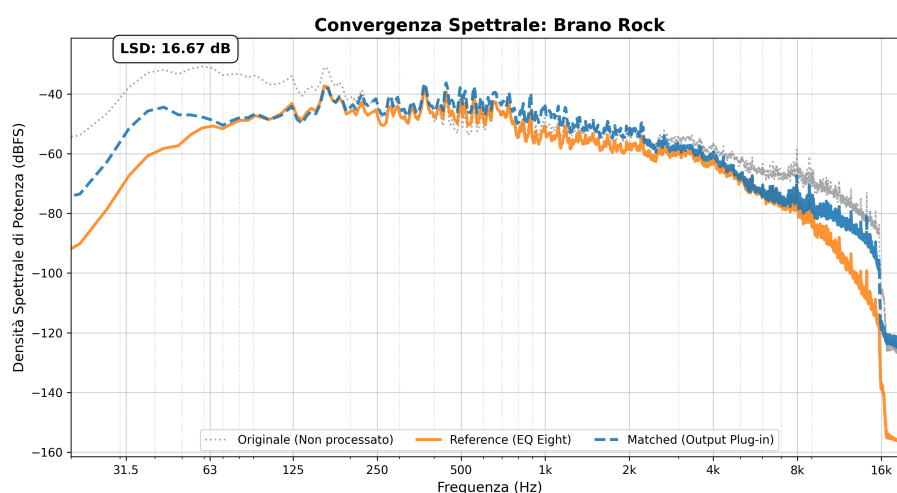


Figura 26: Test 3 - grafico densità di potenza su brano rock

Analisi del risultato: In questo caso, l’LSD si attesta a 16.67 dB. Questo scostamento visibile agli estremi del grafico dipende dalla topologia a filtri usata. Un equalizzatore grafico basato su filtri shelving del secondo ordine non emula perfettamente il decadimento verticale di un filtro passa-alto o passa-basso puro. L’algoritmo compie l’azione più fattibile: portare i fader estremi alla massima attenuazione consentita (-12 dB).

Capitolo 6

Conclusioni e Sviluppi Futuri

6.1 Conclusioni sui risultati ottenuti

Questo elaborato ha illustrato la progettazione, lo sviluppo e la validazione di un plug-in di equalizzazione grafica a 10 bande, dotato di un sistema di matching EQ integrato. L'obiettivo primario di superare la problematica interazione tra bande tipica dei sistemi tradizionali è stato raggiunto grazie all'implementazione della matrice di interazione. Questo approccio rappresenta il principale punto di forza del progetto, poiché garantisce una corrispondenza rigorosa tra il feedback visivo dei cursori e l'effettiva risposta in frequenza, offrendo un controllo molto più diretto e preciso laddove gli equalizzatori classici mancano di un buon riferimento visivo sull'interazione energetica dei filtri. A questa soluzione si affianca l'algoritmo di matching per l'adattamento timbrico automatizzato tra due segnali distinti, una funzionalità oggi estremamente utile e ricercata nell'ambito della produzione musicale, ma che risulta ancora rara da trovare implementata su un equalizzatore grafico.

Da un punto di vista informatico, il codice è stato progettato con un'architettura fortemente modulare. Questa separazione dei compiti rende il software facilmente estendibile e aggiornabile per sviluppi futuri, e permette di mantenere la classe processore principale (e in particolare la sua `processBlock`) estremamente snella. I calcoli matematici e la gestione dei parametri sono stati infatti delegati a moduli esterni, garantendo stabilità ed efficienza nell'elaborazione del segnale audio in tempo reale.

La fase di testing ha evidenziato anche alcune criticità e limitazioni intrinseche del progetto. Nonostante la metrica Log-Spectral Distance (LSD) abbia restituito buoni valori (tra i 2 e i 4 dB), l'approssimazione di involucri spettrali complessi sconta i limiti fisici di un sistema a sole 10 bande. Ad esempio, la topologia dei filtri usati nell'analisi costringe l'algoritmo a un compromesso, generando leggeri scostamenti dalla curva ideale. Inoltre, sempre l'analisi si affida attualmente a un banco di filtri che non mappa in modo del tutto speculare il comportamento dei filtri di equalizzazione. Uno studio più approfondito sulla ricerca di una corrispondenza matematica ancora più coerente tra i filtri di analisi e quelli

di equalizzazione potrebbe ridurre ulteriormente le discrepanze, affinando la precisione del matching.

Nel complesso, i risultati confermano la validità del lavoro svolto. Il plug-in si presenta come uno strumento versatile e, soprattutto, caratterizzato da una grande semplicità di utilizzo. Proprio grazie a questo approccio intuitivo, è una soluzione che trova facilmente spazio nei moderni flussi di lavoro della produzione musicale.

6.2 Sviluppi futuri: prototipazione in ambiente MATLAB con elaborazione Mid/Side e maggior libertà nella scelta delle scale frequenziali

Per esporre dei possibili futuri aggiornamenti del plug-in, viene proposto un prototipo in ambiente MATLAB¹ che esplora nuove funzionalità. La prima estensione testata riguarda il superamento del vincolo delle 10 bande fisse (dettate dallo standard ISO). Nel codice è stata infatti introdotta la funzione:

```
fc = generate_center_frequencies(scale, nBands, fLow, fHigh);
```

dove l'utente potrebbe scegliere quante bande utilizzare, impostare la frequenza di partenza e quella di arrivo, e decidere come distanziarle usando scale diverse (come quella lineare, a semitoni, Mel, Bark o ERB).

Un'altra importante evoluzione presente nel prototipo è l'aggiunta dell'elaborazione Mid/Side. Nello script, i segnali stereo in ingresso (target e reference) vengono infatti prima convertiti in queste due componenti. A questo punto, il motore di matching si sdoppia. L'analisi delle energie e il calcolo della curva lavorano in modo del tutto indipendente per le due componenti, richiamando rispettivamente:

```
create_analysis_filterbank(target_M, Fs_target, fc);
generate_eq_curve(analysis_target_M, analysis_ref_M, 12);
```

Dopo aver applicato l'equalizzazione separata, il segnale viene matematicamente ricombinato nel formato stereo di partenza. Anche se questo prototipo non include ancora la matrice di interazione vista in C++, rappresenta un'ottima base per portare l'elaborazione Mid/Side all'interno del plug-in.

A seguire, l'obiettivo futuro sarà perfezionare l'algoritmo di analisi. Come emerso dalla fase di validazione, l'idea è quella di progettare un banco di filtri di analisi che si comporti in modo ancora più speculare ai filtri di equalizzazione, per rendere il matching sempre più preciso. Infine, è previsto un aggiornamento per l'interfaccia grafica. Attualmente, la linea che unisce i fader offre un riscontro visivo morbido e dal design più

¹Codice sorgente del prototipo MATLAB disponibile al link: <https://github.com/alezampie/Matching-Graphic-EQ-Plugin/tree/main/Matlab-Prototypes>

creativo. Tuttavia, è già in sviluppo la possibilità di mostrare a schermo anche la vera risposta in frequenza dei filtri in cascata (se l'utente preferisce). Questo permetterà all'utente di scegliere liberamente tra un'interfaccia moderna e pulita, e una visualizzazione tecnica e rigorosa del reale comportamento acustico del sistema (Figura 27).

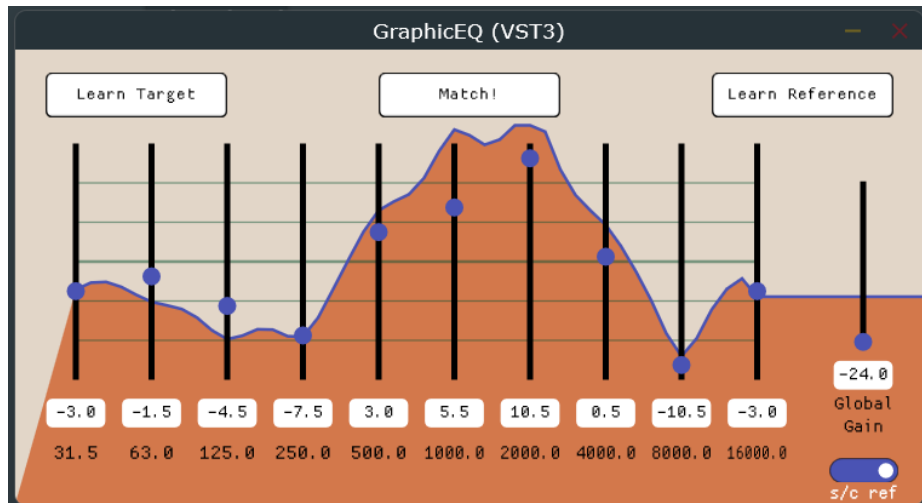


Figura 27: Prototipo di visualizzazione della reale curva di equalizzazione rispetto al posizionamento dei fader.

Bibliografia

- [1] R. Izhaki. *Mixing Audio: Concepts, Practices and Tools*. Focal Press, Oxford, 1st edition, 2008.
- [2] S. Hafezi and J. D. Reiss. Autonomous multitrack equalization based on masking reduction. *Journal of the Audio Engineering Society*, 63(5):312–323, 2015.
- [3] V. Välimäki and J. D. Reiss. All about audio equalization: Solutions and frontiers. *Applied Sciences*, 6(5):1–46, 2016.
- [4] J. S. Abel and D. P. Berners. Filter design using second-order peaking and shelving sections. In *Proceedings of the 116th Convention of the Audio Engineering Society*, Berlin, Germany, 2004. Audio Engineering Society.
- [5] E. Perez-Gonzalez and J. D. Reiss. Automatic equalization of multi-channel audio using cross-adaptive methods. In *Proceedings of the 127th Convention of the Audio Engineering Society*, New York, NY, 2009. Audio Engineering Society.
- [6] A. Leyshon. The software slump?: digital music, the democratisation of technology, and the decline of the recording studio sector within the musical economy. *Environment and Planning A*, 41(6):1309–1331, 2009.
- [7] Waves Audio Ltd. *Waves API 560 User Manual*.
- [8] Waves Audio Ltd. *Waves GEQ Graphic Equalizer User Guide*.
- [9] FabFilter. *FabFilter Pro-Q 3 User Manual, Sezione: "EQ Match"*.
- [10] Native Instruments / iZotope. *Ozone 11 Help Documentation, Sezione: "Match EQ"*.
- [11] D. M. Huber and R. E. Runstein. *Modern Recording Techniques*. Routledge, New York, 9th edition, 2017.
- [12] JUCE. Discover juce: The framework for audio applications. <https://juce.com/discover/>.

- [13] JUCE. Tutorial: Create a basic audio plug-in. https://docs.juce.com/master/tutorial_create_producer_basic_plugin.html.
- [14] JUCE. Juce api reference: juce_dsp module. https://docs.juce.com/master/group__juce__dsp.html.
- [15] Eigen. Eigen - c++ template library for linear algebra. <https://eigen.tuxfamily.org>.
- [16] V. Zavalishin. *The Art of VA Filter Design*. Native Instruments, Berlin, rev. 2.1.0 edition, 2018.
- [17] P. Virtanen, R. Gommers, T. E. Oliphant, et al. Scipy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.
- [18] P. Welch. The use of fast fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.
- [19] A. H. Gray and J. D. Markel. Distance measures for speech processing. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 24(5):380–391, 1976.



Progetto sviluppato presso il Laboratorio di Informatica Musicale (LIM)
<https://www.lim.di.unimi.it>